Final

August 9, 2026

1 Composer Classification of MIDI Music with LSTM and CNN Models

Deep Learning Final Project

This notebook develops and compares two deep learning models - a **Long Short-Term Memory (LSTM)** network and a **Convolutional Neural Network (CNN)** - that predict the composer of a MIDI musical composition. The task is restricted to four composers:

1. **Bach**
2. **Beethoven**
3. **Chopin**
4. **Mozart**

1.1 Project overview

Each architecture receives a representation that suits it. The LSTM consumes *ordered note-pitch sequences* (an event stream), which matches its strength at modeling temporal order. The CNN consumes *piano-roll matrices* (a 2-D pitch-by-time image), which matches its strength at detecting local 2-D patterns such as chords and textures. Both models are trained, tuned, and evaluated on the same train/validation/test partition, and their results are compared directly.

1.2 How to run this notebook

Local Jupyter: 1. Make sure the `dataset_4composers` folder is available at `build/datasets/dataset_4composers` (relative to this notebook). It must contain one sub-folder per composer: `Bach/`, `Beethoven/`, `Chopin/`, `Mozart/`. 2. Run the *Setup* cells to install libraries. 3. Run the notebook top to bottom.

Everything is CPU-compatible; a GPU only speeds up training.

> **Reproducibility & honesty note.** All metrics, tables, and figures in this notebook are produced by the code at run time. Nothing is hard-coded. Cells that print final numbers are marked so you can copy the printed values into the report. If a step cannot run (for example, the dataset is missing), the code fails loudly with a clear message rather than silently continuing.

1.3 1. Setup: install and import libraries

The next cell installs the third-party libraries the notebook needs (`pretty_midi` and `music21` are usually not pre-installed). The install cell is safe to re-run.

[26]:
```
# Must run before any TensorFlow import so env vars and CUDA libs are in place
import os, sys
```

```
# Prepend notebook directory so utilsTF.py is importable
sys.path.insert(0, os.path.dirname(os.path.abspath("FirstPass.ipynb")))

from utilsTF import load_cuda_libraries, configure_tensorflow

load_cuda_libraries()      # pre-load nvidia .so files into process memory
configure_tensorflow()     # set XLA_FLAGS and TF_CPP_MIN_LOG_LEVEL before TF loads
```

```
Loaded CUDA libraries from /home/maxim/miniconda3/envs/511-team-
project-2/lib/python3.11/site-packages/nvidia
```

[27]:
```
from utilsTF import test_gpu

test_gpu()   # enable memory growth and report available GPUs
```

```
GPU setup complete - 1 GPU(s) available
```

[27]:
```
1
```

[28]:
```
# --- Imports ---
import os, re, glob, hashlib, random, warnings, collections
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# MIDI parsing (with graceful failure if unavailable)
try:
    import pretty_midi
except ImportError as e:
    raise ImportError(
        "pretty_midi is required. Run the install cell above, then restart the "
        "runtime and re-run."
    ) from e

# Deep learning
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks, optimizers

# Metrics
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report, confusion_matrix,
                             precision_recall_fscore_support, accuracy_score)
from sklearn.utils.class_weight import compute_class_weight

warnings.filterwarnings("ignore")
pretty_midi.pretty_midi.MAX_TICK = 1e7  # tolerate long files without warnings
```

```
sns.set_context("notebook")
print("TensorFlow version:", tf.__version__)
print("GPU available:", bool(tf.config.list_physical_devices("GPU")))
```

TensorFlow version: 2.16.1
GPU available: True

1.4 2. Configuration and random seeds

All tunable constants live here so the notebook has a single control panel. DATASET_PATH is the **only** path you normally need to change. Random seeds are fixed for NumPy, Python, and TensorFlow so runs are reproducible (small non-determinism can still occur on GPU).

[29]:
```
# ---------------------------- CONFIG ----------------------------------
# dataset_4composers: flat layout with one sub-folder per composer.
# Update this path to wherever dataset_4composers lives on your machine.
DATASET_PATH = "build/datasets/dataset_4composers"

TARGET_COMPOSERS = ["Bach", "Beethoven", "Chopin", "Mozart"]

# LSTM (note-sequence) settings
SEQ_LEN      = 100      # notes per training window
SEQ_STRIDE   = 50       # hop between windows (controls overlap / augmentation)
PITCH_VOCAB  = 128      # MIDI pitches 0-127
PAD_TOKEN    = 128      # reserved padding index -> embedding input_dim = 129

# CNN (piano-roll) settings
PR_FS        = 8        # piano-roll sampling frequency (frames per second)
PR_TIME      = 128      # time frames per window
PR_PITCH     = 128      # pitch rows (fixed by MIDI)
PR_STRIDE    = 64       # hop between piano-roll windows

# Split / training settings
TEST_SIZE    = 0.15
VAL_SIZE     = 0.15     # fraction of the WHOLE dataset (taken from train remainder)
BATCH_SIZE   = 64
MAX_EPOCHS   = 60
PATIENCE     = 8
SEED         = 42

# Data-augmentation (training split only)
AUG_TRANSPOSITIONS = [-2, -1, 1, 2]   # semitone shifts; [] disables transposition

# Cap on windows per file (keeps very long pieces from dominating a split).
# There is no per-composer FILE cap: windows are stored unaugmented as
# int16/uint8 and transposition is applied on the fly in a tf.data pipeline
# (Section 6.3) instead of being pre-computed as extra float32 copies. That is
# what makes the full corpus (~1,600 usable files) fit in RAM without capping
# files per composer.
MAX_WINDOWS_PER_FILE = 20
```

```
# Output directories — created automatically on first run.
ARTIFACTS_DIR      = Path("build/artifacts")       # model checkpoints + training histories
VISUALIZATIONS_DIR = Path("build/visualizations")  # CSVs and exported figures
CACHE_DIR          = Path("build/cache")           # extracted window arrays, keyed by split
ARTIFACTS_DIR.mkdir(parents=True, exist_ok=True)
VISUALIZATIONS_DIR.mkdir(parents=True, exist_ok=True)
CACHE_DIR.mkdir(parents=True, exist_ok=True)

# ~2x faster LSTM/CNN training on NVIDIA GPUs; harmless elsewhere. Models keep
# their softmax in float32 (see build_lstm/build_cnn) as Keras recommends.
if tf.config.list_physical_devices("GPU"):
    tf.keras.mixed_precision.set_global_policy("mixed_float16")
    print("Mixed precision enabled (float16 compute, float32 softmax).")
# ----------------------------------------------------------------------

def set_seeds(seed=SEED):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed); np.random.seed(seed); tf.random.set_seed(seed)

set_seeds()
LABEL2IDX = {c: i for i, c in enumerate(TARGET_COMPOSERS)}
IDX2LABEL = {i: c for c, i in LABEL2IDX.items()}
print("Label encoding:", LABEL2IDX)
```

```
Mixed precision enabled (float16 compute, float32 softmax).
Label encoding: {'Bach': 0, 'Beethoven': 1, 'Chopin': 2, 'Mozart': 3}
```

1.5 3. Dataset loading

Dataset: *midi-classic-music* by Bohdan Fedorak (Kaggle, 2019) — pre-filtered to four composers and stored locally as dataset_4composers. The directory has a **flat layout**: one sub-folder per composer (Bach/, Beethoven/, Chopin/, Mozart/) containing .mid files directly, with no pre-defined train/val/test split.

The data lives in the repository at build/datasets/dataset_4composers. The cell below verifies the directory layout and reports file counts per composer. The **full corpus** (1,637 files) is used — a memory-safe tf.data pipeline (Section 6.3) removes the need for the per-composer file cap used in the earlier submission.

[30]:
```
# --- Verify the dataset directory layout ----------------------------------
if not os.path.isdir(DATASET_PATH):
    raise FileNotFoundError(
        f"DATASET_PATH does not exist: {DATASET_PATH}\n"
        "Set DATASET_PATH in the CONFIG cell to the folder that contains the "
        "composer sub-folders (Bach/, Beethoven/, Chopin/, Mozart/).")

print(f"Dataset root: {os.path.abspath(DATASET_PATH)}")
print(f"Composers ({len(TARGET_COMPOSERS)}): {TARGET_COMPOSERS}\n")

for comp in TARGET_COMPOSERS:
    comp_dir = Path(DATASET_PATH) / comp
```

```
    if comp_dir.is_dir():
        n = sum(1 for ext in ("*.mid", "*.midi", "*.MID", "*.MIDI")
                for _ in comp_dir.rglob(ext))
        print(f"  {comp:12s}: {n} MIDI files")
    else:
        print(f"  {comp:12s}: folder not found at {comp_dir}")
```

```
Dataset root: /mnt/d/Documents/MachineLearning/TeamProject-
511/build/datasets/dataset_4composers
Composers (4): ['Bach', 'Beethoven', 'Chopin', 'Mozart']

  Bach        : 1024 MIDI files
  Beethoven   : 220 MIDI files
  Chopin      : 136 MIDI files
  Mozart      : 257 MIDI files
```

1.6 4. Dataset inspection, filtering, and label creation

The dataset layout is flat: <composer>/<file>.mid. The code **walks** DATASET_PATH **recursively**, collects every .mid/.midi file, and infers the composer from the file path — a file is kept only if exactly one of the four target composer names appears in its path (the exactly-one-match rule rejects ambiguous files). This step also performs **de-duplication by MD5 hash** and drops files that are unreadable or too short to yield a training window.

[31]:
```
def infer_composer(path, targets=TARGET_COMPOSERS):
    """Return the single matching target composer found in the path, or None."""
    matches = [c for c in targets if c.lower() in str(path).lower()]
    return matches[0] if len(matches) == 1 else None

def collect_midi_files(root, composers=TARGET_COMPOSERS):
    """Recursively walk root, infer composer from path, return one record per file."""
    records = []
    for ext in ("*.mid", "*.midi", "*.MID", "*.MIDI"):
        for f in Path(root).rglob(ext):
            comp = infer_composer(f, composers)
            if comp is not None:
                records.append({"path": str(f), "composer": comp})
    return (pd.DataFrame(records)
              .drop_duplicates(subset="path")
              .sort_values("path")
              .reset_index(drop=True))

raw_df = collect_midi_files(DATASET_PATH)
if len(raw_df) == 0:
    raise RuntimeError("No MIDI files found. Check DATASET_PATH and the dataset contents.")

print(f"Files matching the four target composers: {len(raw_df)}")
print("\nRaw counts per composer (before cleaning):")
print(raw_df["composer"].value_counts())
```

Files matching the four target composers: 1637

Raw counts per composer (before cleaning):
composer
Bach 1024
Mozart 257
Beethoven 220
Chopin 136
Name: count, dtype: int64

[32]:
```
# --- De-duplicate by content hash and drop unreadable files ----------
def md5_of(path, chunk=1 << 20):
    h = hashlib.md5()
    with open(path, "rb") as fh:
        for block in iter(lambda: fh.read(chunk), b""):
            h.update(block)
    return h.hexdigest()

raw_df["md5"] = raw_df["path"].map(md5_of)
before = len(raw_df)
dedup_df = raw_df.drop_duplicates(subset="md5").reset_index(drop=True)
print(f"Duplicate files removed: {before - len(dedup_df)}")

# Validate that each file is parseable; record note counts.
def probe_midi(path):
    try:
        pm = pretty_midi.PrettyMIDI(path)
        n_notes = sum(len(inst.notes) for inst in pm.instruments)
        return n_notes
    except Exception:
        return -1   # unreadable / invalid

dedup_df["n_notes"] = dedup_df["path"].map(probe_midi)
unreadable = int((dedup_df["n_notes"] < 0).sum())
empty      = int((dedup_df["n_notes"] == 0).sum())
print(f"Unreadable / invalid MIDI files: {unreadable}")
print(f"Readable but empty (no notes):   {empty}")

# Keep only files with a usable number of notes for at least one window.
clean_df = dedup_df[dedup_df["n_notes"] >= SEQ_LEN].reset_index(drop=True)
print(f"\nUsable files (>= {SEQ_LEN} notes): {len(clean_df)}")
print("\nUsable counts per composer (after cleaning):")
print(clean_df["composer"].value_counts())
```

Duplicate files removed: 21
Unreadable / invalid MIDI files: 2
Readable but empty (no notes): 0

Usable files (>= 100 notes): 1611

Usable counts per composer (after cleaning):
composer
Bach 1012
Mozart 254
Beethoven 213
Chopin 132
Name: count, dtype: int64

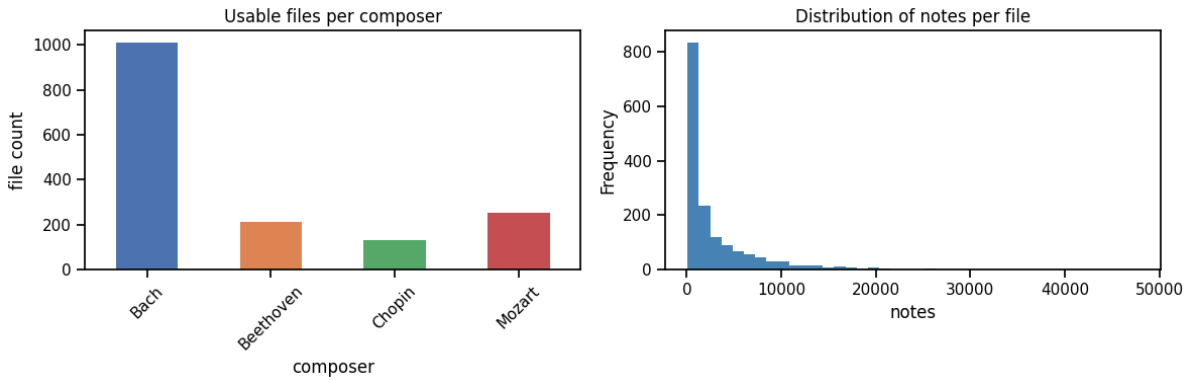
1.6.1 4.1 Before/after summary and class distribution

The table below reports usable file counts per composer **before and after** preprocessing, satisfying the requirement to document data availability explicitly. The bar chart visualizes class balance; any strong imbalance is addressed later with class weights during training.

[33]:
```
summary = (pd.DataFrame({
        "raw_matched": raw_df["composer"].value_counts(),
        "after_dedup": dedup_df["composer"].value_counts(),
        "usable":      clean_df["composer"].value_counts(),
    })
    .reindex(TARGET_COMPOSERS)
    .fillna(0).astype(int))
summary.loc["TOTAL"] = summary.sum()
print("File counts per composer:\n")
print(summary)

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
clean_df["composer"].value_counts().reindex(TARGET_COMPOSERS).plot(
    kind="bar", ax=ax[0], color=sns.color_palette("deep", len(TARGET_COMPOSERS)))
ax[0].set_title("Usable files per composer"); ax[0].set_ylabel("file count")
ax[0].tick_params(axis="x", rotation=45)
clean_df["n_notes"].plot(kind="hist", bins=40, ax=ax[1], color="steelblue")
ax[1].set_title("Distribution of notes per file"); ax[1].set_xlabel("notes")
plt.tight_layout(); plt.show()
```

File counts per composer:

composer	raw_matched	after_dedup	usable
Bach	1024	1014	1012
Beethoven	220	215	213
Chopin	136	132	132
Mozart	257	255	254
TOTAL	1637	1616	1611



1.7 5. Train / validation / test splitting (leakage-safe)

The split is performed **at the file level, before any windowing**, using a stratified 70/15/15 train/val/test partition over the **full corpus** (no per-composer file cap). Splitting whole files first guarantees that every window from a given composition stays inside a single split — windows from one piece can never land in both training and test sets, which would let the model memorize a piece rather than a composer's style. The MD5 deduplication from Section 4 additionally guarantees that no identical file appears in two splits, and the assertion below verifies it.

[34]:
```
files_all = clean_df["path"].values
labels_all = clean_df["composer"].map(LABEL2IDX).values

# Stratified split: first carve out test, then split remainder into train/val.
f_trainval, f_test, y_trainval, y_test = train_test_split(
    files_all, labels_all, test_size=TEST_SIZE, stratify=labels_all, random_state=SEED)
val_relative = VAL_SIZE / (1.0 - TEST_SIZE)
f_train, f_val, y_train, y_val = train_test_split(
    f_trainval, y_trainval, test_size=val_relative,
    stratify=y_trainval, random_state=SEED)

print(f"Files -> train: {len(f_train)}  val: {len(f_val)}  test: {len(f_test)}")
for name, y in [("train", y_train), ("val", y_val), ("test", y_test)]:
    dist = {IDX2LABEL[i]: int((y == i).sum()) for i in range(len(TARGET_COMPOSERS))}
    print(f"  {name:5s} class counts: {dist}")

# Sanity check: no file appears in more than one split.
assert set(f_train) & set(f_test) == set()
assert set(f_train) & set(f_val)  == set()
assert set(f_val)   & set(f_test) == set()
print("Leakage check passed: splits are disjoint at the file level.")
```

```
Files -> train: 1127  val: 242  test: 242
  train class counts: {'Bach': 708, 'Beethoven': 149, 'Chopin': 92, 'Mozart':
178}
  val   class counts: {'Bach': 152, 'Beethoven': 32, 'Chopin': 20, 'Mozart': 38}
  test  class counts: {'Bach': 152, 'Beethoven': 32, 'Chopin': 20, 'Mozart': 38}
```

Leakage check passed: splits are disjoint at the file level.

1.8 6. Feature extraction

Two representations are extracted from the **same** MIDI files, one tailored to each model. Both come from a **single parse** of each file — parsing dominates extraction time, so parsing every file twice across the full corpus would roughly double this step's runtime.

LSTM - note-pitch sequences. Every note across all instruments is sorted by onset time, and its MIDI pitch (0-127) becomes a token, stored as int16. The stream is cut into overlapping windows of SEQ_LEN notes (hop SEQ_STRIDE). Ordered pitch tokens preserve melodic and harmonic motion over time, which is what a recurrent network is built to model. An Embedding layer turns each pitch token into a learned dense vector.

CNN - piano-roll windows. pretty_midi.get_piano_roll(fs=PR_FS) produces a 128 x time matrix of note activity. It is binarized, stored as uint8, and cut into PR_PITCH x PR_TIME windows (hop PR_STRIDE). A piano roll is effectively an image of the music, so 2-D convolutions can pick up chords (vertical structure) and rhythmic/melodic motifs (horizontal structure).

Because both representations are windowed, one file yields several samples - a form of **sequence-window augmentation** that also enlarges the dataset. Each window also carries the id of its source file, so test-set predictions can later be aggregated back to piece level (Section 12).

[35]:
```
def extract_both(path):
    """Parse a MIDI file once and derive both representations from it.
    Returns (seq, roll); either is None if that representation isn't usable."""
    try:
        pm = pretty_midi.PrettyMIDI(path)
    except Exception:
        return None, None
    notes = [(n.start, n.pitch) for inst in pm.instruments
             if not inst.is_drum for n in inst.notes]
    seq = None
    if len(notes) >= SEQ_LEN:
        notes.sort(key=lambda x: x[0])
        seq = np.array([p for _, p in notes], dtype=np.int16)
    roll = None
    if len(pm.instruments) > 0:
        r = pm.get_piano_roll(fs=PR_FS)          # (128, T) velocity
        if r.shape[1] >= PR_TIME:
            roll = (r > 0).astype(np.uint8)      # binarize
    return seq, roll

def sequence_windows(seq, seq_len=SEQ_LEN, stride=SEQ_STRIDE,
    ↪cap=MAX_WINDOWS_PER_FILE):
    out = []
    for start in range(0, len(seq) - seq_len + 1, stride):
        out.append(seq[start:start + seq_len])
        if cap is not None and len(out) >= cap:
            break
    return out
```

```
def roll_windows(roll, t=PR_TIME, stride=PR_STRIDE, cap=MAX_WINDOWS_PER_FILE):
    out = []
    for start in range(0, roll.shape[1] - t + 1, stride):
        out.append(roll[:, start:start + t])
        if cap is not None and len(out) >= cap:
            break
    return out

print("Feature-extraction helpers defined.")
```

Feature-extraction helpers defined.

1.8.1 6.1 Build windowed datasets per split

The helper below turns a list of files into arrays of **unaugmented** windows plus matching labels and file ids, with results cached to disk per split so re-running the notebook doesn't re-parse the full corpus. Augmentation (pitch transposition) is applied to the training split only, and only inside the tf.data pipeline built in 6.3 — not here — so validation and test windows are guaranteed to never be augmented, preserving an honest estimate of generalization. Transposition by a few semitones changes key but preserves intervals and contour, so the composer's style is retained.

[36]:
```
def build_split_arrays(files, labels):
    """One row per window; base (unaugmented) windows only, with a file id.
    Storing base windows instead of pre-transposing 5 float32 copies per
    window (as the file-capped version did) is what lets the full corpus fit
    in RAM; the 5 copies are instead sampled on the fly in 6.3."""
    seq_X, seq_y, seq_fid, skipped_seq = [], [], [], 0
    cnn_X, cnn_y, cnn_fid, skipped_cnn = [], [], [], 0
    for fid, (path, lab) in enumerate(zip(files, labels)):
        seq, roll = extract_both(path)
        if seq is None:
            skipped_seq += 1
        else:
            for w in sequence_windows(seq):
                seq_X.append(w); seq_y.append(lab); seq_fid.append(fid)
        if roll is None:
            skipped_cnn += 1
        else:
            for w in roll_windows(roll):
                cnn_X.append(w); cnn_y.append(lab); cnn_fid.append(fid)
    Xseq = np.array(seq_X, dtype=np.int16)
    Xcnn = np.array(cnn_X, dtype=np.uint8)[..., np.newaxis]   # add channel dim
    return {
        "seq": (Xseq, np.array(seq_y, dtype=np.int32), np.array(seq_fid, dtype=np.int64),␣
↪skipped_seq),
        "cnn": (Xcnn, np.array(cnn_y, dtype=np.int32), np.array(cnn_fid, dtype=np.int64),␣
↪skipped_cnn),
    }

def cached_split_arrays(split, files, labels):
```

```
    """Extraction is deterministic given (split, config); cache it to disk so a
    kernel restart doesn't re-parse ~1,600 MIDI files. Delete CACHE_DIR if any
    window/extraction parameter in the CONFIG cell changes."""
    p = CACHE_DIR / f"{split}.npz"
    if p.exists():
        z = np.load(p)
        print(f"  [{split}] loaded from cache ({p})")
        return {"seq": (z["sX"], z["sy"], z["sfid"], int(z["sskip"])),
                "cnn": (z["cX"], z["cy"], z["cfid"], int(z["cskip"]))}
    out = build_split_arrays(files, labels)
    (sX, sy, sfid, sskip), (cX, cy, cfid, cskip) = out["seq"], out["cnn"]
    np.savez_compressed(p, sX=sX, sy=sy, sfid=sfid, sskip=sskip,
                        cX=cX, cy=cy, cfid=cfid, cskip=cskip)
    print(f"  [{split}] extracted and cached -> {p}")
    return out

print("Split builders defined.")
```

Split builders defined.

[37]:
```
train_arr = cached_split_arrays("train", f_train, y_train)
val_arr   = cached_split_arrays("val",   f_val,   y_val)
test_arr  = cached_split_arrays("test",  f_test,  y_test)

Xseq_train, yseq_train, seq_train_fid, sk1 = train_arr["seq"]
Xseq_val,   yseq_val,   seq_val_fid,   sk2 = val_arr["seq"]
Xseq_test,  yseq_test,  seq_test_fid,  sk3 = test_arr["seq"]
print("LSTM (note-sequence) windows")
print(f"  train: {Xseq_train.shape}  val: {Xseq_val.shape}  test: {Xseq_test.shape}")
print(f"  files skipped (train/val/test): {sk1}/{sk2}/{sk3}")

Xcnn_train, ycnn_train, cnn_train_fid, sk4 = train_arr["cnn"]
Xcnn_val,   ycnn_val,   cnn_val_fid,   sk5 = val_arr["cnn"]
Xcnn_test,  ycnn_test,  cnn_test_fid,  sk6 = test_arr["cnn"]
print("\nCNN (piano-roll) windows")
print(f"  train: {Xcnn_train.shape}  val: {Xcnn_val.shape}  test: {Xcnn_test.shape}")
print(f"  files skipped (train/val/test): {sk4}/{sk5}/{sk6}")
```

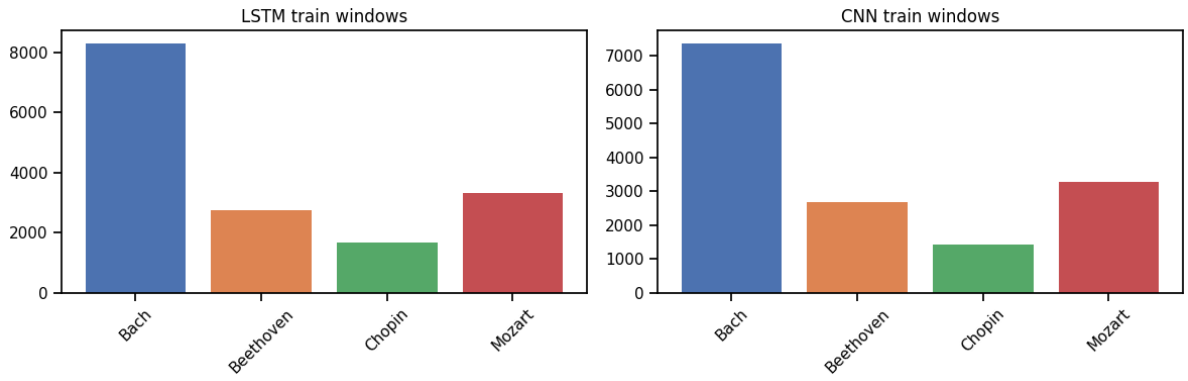
```
  [train] loaded from cache (build/cache/train.npz)
  [val] loaded from cache (build/cache/val.npz)
  [test] loaded from cache (build/cache/test.npz)
LSTM (note-sequence) windows
  train: (16002, 100)  val: (3363, 100)  test: (3326, 100)
  files skipped (train/val/test): 0/0/0

CNN (piano-roll) windows
  train: (14738, 128, 128, 1)  val: (3146, 128, 128, 1)  test: (3011, 128, 128,
1)
  files skipped (train/val/test): 0/0/0
```

1.8.2 6.2 Window-level class distribution and class weights

Windowing can change class balance because some composers have longer pieces (more windows). The counts below reflect the **actual training signal**. Class weights are computed from the training windows and passed to both models so that minority composers are not ignored.

[38]:
```
def show_distribution(y, title):
    counts = {IDX2LABEL[i]: int((y == i).sum()) for i in range(len(TARGET_COMPOSERS))}
    print(f"{title}: {counts}")
    return counts

print("Window counts per class")
_ = show_distribution(yseq_train, "LSTM train")
_ = show_distribution(ycnn_train, "CNN  train")

# Class weights (shared logic; computed separately per representation)
def class_weights(y):
    classes = np.arange(len(TARGET_COMPOSERS))
    w = compute_class_weight("balanced", classes=classes, y=y)
    return {int(c): float(wi) for c, wi in zip(classes, w)}

cw_seq = class_weights(yseq_train)
cw_cnn = class_weights(ycnn_train)
print("\nLSTM class weights:", {IDX2LABEL[k]: round(v, 3) for k, v in cw_seq.items()})
print("CNN  class weights:", {IDX2LABEL[k]: round(v, 3) for k, v in cw_cnn.items()})

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
for a, (y, t) in zip(ax, [(yseq_train, "LSTM train windows"),
                          (ycnn_train, "CNN train windows")]):
    vals = [ (y == i).sum() for i in range(len(TARGET_COMPOSERS)) ]
    a.bar(TARGET_COMPOSERS, vals, color=sns.color_palette("deep",␣
 ↪len(TARGET_COMPOSERS)))
    a.set_title(t); a.tick_params(axis="x", rotation=45)
plt.tight_layout(); plt.show()
```

```
Window counts per class
LSTM train: {'Bach': 8299, 'Beethoven': 2732, 'Chopin': 1666, 'Mozart': 3305}
CNN  train: {'Bach': 7371, 'Beethoven': 2679, 'Chopin': 1428, 'Mozart': 3260}

LSTM class weights: {'Bach': 0.482, 'Beethoven': 1.464, 'Chopin': 2.401,
'Mozart': 1.21}
CNN  class weights: {'Bach': 0.5, 'Beethoven': 1.375, 'Chopin': 2.58, 'Mozart':
1.13}
```



1.8.3 6.3 tf.data pipelines with on-the-fly augmentation

The arrays above hold one **unaugmented** window per row. Instead of pre-computing four transposed copies per training window (a 5x memory multiplier that forced the earlier per-composer file cap), each training window is randomly transposed by one of AUG_TRANSPOSITIONS (or left unshifted) **inside the** tf.data **pipeline**, resampled every epoch. This covers the same augmentation space at a fraction of the memory cost, which is what makes training on the full corpus fit in RAM. Validation and test windows are never augmented.

[39]:
```
SHIFTS = tf.constant([0] + AUG_TRANSPOSITIONS, dtype=tf.int32)

def _aug_seq(x, y):
    s = SHIFTS[tf.random.uniform([], 0, len(AUG_TRANSPOSITIONS) + 1, tf.int32)]
    x = tf.clip_by_value(tf.cast(x, tf.int32) + s, 0, PITCH_VOCAB - 1)
    return x, y

def _shift_roll(x, s):
    # x: (PR_PITCH, PR_TIME, 1); row index = pitch, so shifting rows transposes.
    def up():   return tf.pad(x, [[s, 0], [0, 0], [0, 0]])[:PR_PITCH]
    def down(): return tf.pad(x, [[0, -s], [0, 0], [0, 0]])[-PR_PITCH:]
    return tf.cond(s > 0, up, down)

def _aug_roll(x, y):
    s = SHIFTS[tf.random.uniform([], 0, len(AUG_TRANSPOSITIONS) + 1, tf.int32)]
    x = tf.cond(tf.equal(s, 0), lambda: x, lambda: _shift_roll(x, s))
    return x, y

def make_ds(X, y, kind, training):
    if kind == "seq":
        ds = tf.data.Dataset.from_tensor_slices((X.astype(np.int32), y))
        aug = _aug_seq
    else:
        ds = tf.data.Dataset.from_tensor_slices((X.astype(np.float32), y))
        aug = _aug_roll
    if training:
        ds = ds.shuffle(min(len(y), 20000), seed=SEED).map(
```

```
            aug, num_parallel_calls=tf.data.AUTOTUNE)
    return ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

seq_train_ds = make_ds(Xseq_train, yseq_train, "seq", training=True)
seq_val_ds   = make_ds(Xseq_val,   yseq_val,   "seq", training=False)
seq_test_ds  = make_ds(Xseq_test,  yseq_test,  "seq", training=False)

cnn_train_ds = make_ds(Xcnn_train, ycnn_train, "cnn", training=True)
cnn_val_ds   = make_ds(Xcnn_val,   ycnn_val,   "cnn", training=False)
cnn_test_ds  = make_ds(Xcnn_test,  ycnn_test,  "cnn", training=False)

print("tf.data pipelines ready.")
```

```
2026-07-26 01:28:43.268531: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
965869568 exceeds 10% of free system memory.
2026-07-26 01:28:44.404862: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
965869568 exceeds 10% of free system memory.
```

```
tf.data pipelines ready.
```

1.9 7. Model 1 - LSTM

The LSTM classifies a window of ordered pitch tokens. An Embedding layer maps each of the 129 possible tokens (128 pitches + 1 padding index) to a dense vector; two stacked LSTM layers model temporal dependencies; dropout regularizes; and a softmax Dense layer outputs a probability over the composer classes.

Key hyperparameters: embedding dim 64, LSTM units 128 then 64, dropout 0.3, Adam optimizer, sparse categorical cross-entropy (labels are integers). These are explained in the report.

[40]:
```
def build_lstm(units1=128, units2=64, emb_dim=64, dropout=0.3, lr=1e-3):
    set_seeds()
    model = models.Sequential([
        layers.Input(shape=(SEQ_LEN,)),
        layers.Embedding(input_dim=PITCH_VOCAB + 1, output_dim=emb_dim,
                         mask_zero=False),
        layers.LSTM(units1, return_sequences=True, dropout=dropout),
        layers.LSTM(units2, dropout=dropout),
        layers.Dense(64, activation="relu"),
        layers.Dropout(dropout),
        layers.Dense(len(TARGET_COMPOSERS), activation="softmax", dtype="float32"),
    ], name="LSTM_composer_classifier")
    model.compile(optimizer=optimizers.Adam(lr),
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    return model

build_lstm().summary()
```

```
Model: "LSTM_composer_classifier"
```

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 100, 64)	8,256
lstm_2 (LSTM)	(None, 100, 128)	98,816
lstm_3 (LSTM)	(None, 64)	49,408
dense_4 (Dense)	(None, 64)	4,160
dropout_2 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 4)	260

Total params: 160,900 (628.52 KB)

Trainable params: 160,900 (628.52 KB)

Non-trainable params: 0 (0.00 B)

1.10 8. Model 2 - CNN

The CNN classifies a piano-roll window (128 pitches x PR_TIME frames x 1 channel). Three convolutional blocks (Conv -> BatchNorm -> MaxPool) grow the receptive field; global average pooling collapses the feature map; and a softmax Dense layer outputs the composer probabilities. Batch normalization and dropout provide regularization.

Key hyperparameters: filters 32 -> 64 -> 128, 3x3 kernels, dropout 0.3, Adam optimizer, sparse categorical cross-entropy.

[41]:
```
def build_cnn(f1=32, f2=64, f3=128, dropout=0.3, lr=1e-3):
    set_seeds()
    model = models.Sequential([
        layers.Input(shape=(PR_PITCH, PR_TIME, 1)),
        layers.Conv2D(f1, 3, padding="same", activation="relu"),
        layers.BatchNormalization(), layers.MaxPooling2D(2),
        layers.Conv2D(f2, 3, padding="same", activation="relu"),
        layers.BatchNormalization(), layers.MaxPooling2D(2),
        layers.Conv2D(f3, 3, padding="same", activation="relu"),
        layers.BatchNormalization(), layers.MaxPooling2D(2),
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation="relu"),
        layers.Dropout(dropout),
        layers.Dense(len(TARGET_COMPOSERS), activation="softmax", dtype="float32"),
```

```
    ], name="CNN_composer_classifier")
    model.compile(optimizer=optimizers.Adam(lr),
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    return model

build_cnn().summary()
```

Model: "CNN_composer_classifier"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 128, 128, 32)	320
batch_normalization_3 (BatchNormalization)	(None, 128, 128, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_4 (Conv2D)	(None, 64, 64, 64)	18,496
batch_normalization_4 (BatchNormalization)	(None, 64, 64, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_5 (Conv2D)	(None, 32, 32, 128)	73,856
batch_normalization_5 (BatchNormalization)	(None, 32, 32, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 16, 16, 128)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 128)	0
dense_6 (Dense)	(None, 128)	16,512
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 4)	516

Total params: 110,596 (432.02 KB)

Trainable params: 110,148 (430.27 KB)

Non-trainable params: 448 (1.75 KB)

1.11 9. Training with callbacks

Both models use the same training routine: **early stopping** on validation loss (restores the best weights) and **model checkpointing** to save the best model. Class weights address imbalance. Training and validation loss/accuracy are recorded for later plotting.

[42]:
```
def train_model(build_fn, tr_ds, va_ds, class_weight, tag, **kw):
    set_seeds()
    model = build_fn(**kw)
    ckpt_path = str(ARTIFACTS_DIR / f"best_{tag}.keras")
    cbs = [
        callbacks.EarlyStopping(monitor="val_loss", patience=PATIENCE,
                                restore_best_weights=True),
        callbacks.ModelCheckpoint(ckpt_path, monitor="val_loss",
                                  save_best_only=True),
        callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
                                    patience=3, min_lr=1e-5),
    ]
    history = model.fit(
        tr_ds, validation_data=va_ds,
        epochs=MAX_EPOCHS,
        class_weight=class_weight, callbacks=cbs, verbose=2)
    # Save history alongside the model so training can be skipped later.
    import json as _json
    with open(ARTIFACTS_DIR / f"history_{tag}.json", "w") as fh:
        _json.dump(history.history, fh)
    return model, history

print("Training utility ready.")
```

Training utility ready.

1.12 10. Hyperparameter optimization

A small, honest grid search is run for each model over a few settings (learning rate and dropout). The configuration with the best **validation accuracy** is kept and later evaluated on the untouched test set. The grid is intentionally small so it finishes in a reasonable time; widen it if you have more compute.

> The printed table is produced at run time - copy its real values into the report's optimization table. No numbers are pre-filled.

[43]:
```
LSTM_GRID = [
    {"lr": 1e-3, "dropout": 0.3},
    {"lr": 5e-4, "dropout": 0.3},
    {"lr": 1e-3, "dropout": 0.5},
]
CNN_GRID = [
    {"lr": 1e-3, "dropout": 0.3},
```

```
    {"lr": 5e-4, "dropout": 0.3},
    {"lr": 1e-3, "dropout": 0.5},
]

def grid_search(build_fn, grid, tr_ds, va_ds, cw, tag):
    """Run grid search, skipping any config whose checkpoint already exists."""
    import json as _json
    rows, best = [], None
    for i, params in enumerate(grid):
        cfg_tag      = f"{tag}_cfg{i}"
        model_path   = ARTIFACTS_DIR / f"best_{cfg_tag}.keras"
        history_path = ARTIFACTS_DIR / f"history_{cfg_tag}.json"
        if model_path.exists():
            print(f"[{cfg_tag}] checkpoint found, loading from disk ...")
            model = tf.keras.models.load_model(str(model_path))
            if history_path.exists():
                with open(history_path) as fh:
                    hist_data = _json.load(fh)
            else:
                hist_data = {"val_accuracy": [0], "accuracy": [0],
                             "val_loss": [], "loss": []}
            class _H:
                history = hist_data
            hist = _H()
        else:
            model, hist = train_model(build_fn, tr_ds, va_ds, cw,
                                      tag=cfg_tag, **params)
        val_acc = max(hist.history["val_accuracy"])
        rows.append({**params, "best_val_acc": round(val_acc, 4)})
        if best is None or val_acc > best["val_acc"]:
            best = {"val_acc": val_acc, "params": params,
                    "model": model, "history": hist}
    table = pd.DataFrame(rows)
    print(f"\n{tag} grid-search results:\n", table)
    print(f"Best {tag} config: {best['params']} "
          f"(val_acc={best['val_acc']:.4f})")
    return best, table
```

[44]:
```
# ---- Run LSTM search ----
best_lstm, lstm_grid_table = grid_search(
    build_lstm, LSTM_GRID, seq_train_ds, seq_val_ds, cw_seq, tag="lstm")
lstm_model, lstm_history = best_lstm["model"], best_lstm["history"]
```

```
Epoch 1/60
251/251 - 20s - 78ms/step - accuracy: 0.4194 - loss: 1.3561 - val_accuracy:
0.2403 - val_loss: 1.5324 - learning_rate: 0.0010
Epoch 2/60
251/251 - 8s - 31ms/step - accuracy: 0.4859 - loss: 1.3313 - val_accuracy:
0.4205 - val_loss: 1.3789 - learning_rate: 0.0010
Epoch 3/60
```

251/251 - 8s - 31ms/step - accuracy: 0.4860 - loss: 1.3325 - val_accuracy:
0.5028 - val_loss: 1.2454 - learning_rate: 0.0010
Epoch 4/60
251/251 - 11s - 44ms/step - accuracy: 0.5064 - loss: 1.3151 - val_accuracy:
0.5819 - val_loss: 1.1600 - learning_rate: 0.0010
Epoch 5/60
251/251 - 9s - 35ms/step - accuracy: 0.4951 - loss: 1.3167 - val_accuracy:
0.5471 - val_loss: 1.3070 - learning_rate: 0.0010
Epoch 6/60
251/251 - 10s - 39ms/step - accuracy: 0.4868 - loss: 1.3485 - val_accuracy:
0.4561 - val_loss: 1.4358 - learning_rate: 0.0010
Epoch 7/60
251/251 - 13s - 51ms/step - accuracy: 0.4985 - loss: 1.3219 - val_accuracy:
0.5697 - val_loss: 1.1858 - learning_rate: 0.0010
Epoch 8/60
251/251 - 10s - 39ms/step - accuracy: 0.5060 - loss: 1.2955 - val_accuracy:
0.5947 - val_loss: 1.1477 - learning_rate: 5.0000e-04
Epoch 9/60
251/251 - 8s - 30ms/step - accuracy: 0.5253 - loss: 1.2809 - val_accuracy:
0.5706 - val_loss: 1.1826 - learning_rate: 5.0000e-04
Epoch 10/60
251/251 - 9s - 37ms/step - accuracy: 0.5102 - loss: 1.2797 - val_accuracy:
0.5795 - val_loss: 1.1430 - learning_rate: 5.0000e-04
Epoch 11/60
251/251 - 12s - 46ms/step - accuracy: 0.5112 - loss: 1.2838 - val_accuracy:
0.5754 - val_loss: 1.1351 - learning_rate: 5.0000e-04
Epoch 12/60
251/251 - 7s - 30ms/step - accuracy: 0.5214 - loss: 1.2565 - val_accuracy:
0.5864 - val_loss: 1.1120 - learning_rate: 5.0000e-04
Epoch 13/60
251/251 - 8s - 30ms/step - accuracy: 0.5227 - loss: 1.2430 - val_accuracy:
0.6004 - val_loss: 1.0927 - learning_rate: 5.0000e-04
Epoch 14/60
251/251 - 8s - 31ms/step - accuracy: 0.5301 - loss: 1.2274 - val_accuracy:
0.5822 - val_loss: 1.0757 - learning_rate: 5.0000e-04
Epoch 15/60
251/251 - 12s - 48ms/step - accuracy: 0.5379 - loss: 1.2183 - val_accuracy:
0.5965 - val_loss: 1.0737 - learning_rate: 5.0000e-04
Epoch 16/60
251/251 - 10s - 39ms/step - accuracy: 0.5465 - loss: 1.2062 - val_accuracy:
0.5804 - val_loss: 1.0842 - learning_rate: 5.0000e-04
Epoch 17/60
251/251 - 8s - 32ms/step - accuracy: 0.5410 - loss: 1.2058 - val_accuracy:
0.5674 - val_loss: 1.0908 - learning_rate: 5.0000e-04
Epoch 18/60
251/251 - 7s - 28ms/step - accuracy: 0.5470 - loss: 1.1914 - val_accuracy:
0.5965 - val_loss: 1.0522 - learning_rate: 5.0000e-04
Epoch 19/60
251/251 - 10s - 40ms/step - accuracy: 0.5564 - loss: 1.1730 - val_accuracy:
0.6066 - val_loss: 1.0151 - learning_rate: 5.0000e-04

Epoch 20/60
251/251 - 8s - 31ms/step - accuracy: 0.5589 - loss: 1.1712 - val_accuracy:
0.6063 - val_loss: 1.0311 - learning_rate: 5.0000e-04
Epoch 21/60
251/251 - 8s - 32ms/step - accuracy: 0.5735 - loss: 1.1583 - val_accuracy:
0.6188 - val_loss: 0.9794 - learning_rate: 5.0000e-04
Epoch 22/60
251/251 - 8s - 31ms/step - accuracy: 0.5646 - loss: 1.1583 - val_accuracy:
0.6221 - val_loss: 0.9742 - learning_rate: 5.0000e-04
Epoch 23/60
251/251 - 11s - 44ms/step - accuracy: 0.5802 - loss: 1.1386 - val_accuracy:
0.6099 - val_loss: 1.0234 - learning_rate: 5.0000e-04
Epoch 24/60
251/251 - 8s - 31ms/step - accuracy: 0.5818 - loss: 1.1233 - val_accuracy:
0.6366 - val_loss: 0.9445 - learning_rate: 5.0000e-04
Epoch 25/60
251/251 - 8s - 32ms/step - accuracy: 0.5896 - loss: 1.1106 - val_accuracy:
0.6221 - val_loss: 1.0021 - learning_rate: 5.0000e-04
Epoch 26/60
251/251 - 8s - 33ms/step - accuracy: 0.5998 - loss: 1.0835 - val_accuracy:
0.6482 - val_loss: 0.9158 - learning_rate: 5.0000e-04
Epoch 27/60
251/251 - 11s - 44ms/step - accuracy: 0.6147 - loss: 1.0569 - val_accuracy:
0.6592 - val_loss: 0.9263 - learning_rate: 5.0000e-04
Epoch 28/60
251/251 - 8s - 30ms/step - accuracy: 0.6153 - loss: 1.0533 - val_accuracy:
0.6652 - val_loss: 0.8865 - learning_rate: 5.0000e-04
Epoch 29/60
251/251 - 8s - 30ms/step - accuracy: 0.6269 - loss: 1.0263 - val_accuracy:
0.6304 - val_loss: 0.9581 - learning_rate: 5.0000e-04
Epoch 30/60
251/251 - 8s - 31ms/step - accuracy: 0.6280 - loss: 1.0206 - val_accuracy:
0.6488 - val_loss: 0.9250 - learning_rate: 5.0000e-04
Epoch 31/60
251/251 - 13s - 51ms/step - accuracy: 0.6352 - loss: 0.9935 - val_accuracy:
0.6857 - val_loss: 0.8424 - learning_rate: 5.0000e-04
Epoch 32/60
251/251 - 9s - 34ms/step - accuracy: 0.6429 - loss: 0.9859 - val_accuracy:
0.6729 - val_loss: 0.8554 - learning_rate: 5.0000e-04
Epoch 33/60
251/251 - 8s - 31ms/step - accuracy: 0.6525 - loss: 0.9629 - val_accuracy:
0.6809 - val_loss: 0.8276 - learning_rate: 5.0000e-04
Epoch 34/60
251/251 - 8s - 31ms/step - accuracy: 0.6527 - loss: 0.9490 - val_accuracy:
0.6887 - val_loss: 0.8394 - learning_rate: 5.0000e-04
Epoch 35/60
251/251 - 11s - 44ms/step - accuracy: 0.6557 - loss: 0.9418 - val_accuracy:
0.6902 - val_loss: 0.8205 - learning_rate: 5.0000e-04
Epoch 36/60
251/251 - 9s - 35ms/step - accuracy: 0.6602 - loss: 0.9299 - val_accuracy:

0.6530 - val_loss: 0.9143 - learning_rate: 5.0000e-04
Epoch 37/60
251/251 - 8s - 30ms/step - accuracy: 0.6653 - loss: 0.9142 - val_accuracy:
0.6896 - val_loss: 0.8257 - learning_rate: 5.0000e-04
Epoch 38/60
251/251 - 8s - 31ms/step - accuracy: 0.6704 - loss: 0.8989 - val_accuracy:
0.6812 - val_loss: 0.8510 - learning_rate: 5.0000e-04
Epoch 39/60
251/251 - 11s - 44ms/step - accuracy: 0.6853 - loss: 0.8740 - val_accuracy:
0.6866 - val_loss: 0.8320 - learning_rate: 2.5000e-04
Epoch 40/60
251/251 - 8s - 30ms/step - accuracy: 0.6851 - loss: 0.8622 - val_accuracy:
0.6958 - val_loss: 0.8331 - learning_rate: 2.5000e-04
Epoch 41/60
251/251 - 9s - 34ms/step - accuracy: 0.6894 - loss: 0.8555 - val_accuracy:
0.6866 - val_loss: 0.8355 - learning_rate: 2.5000e-04
Epoch 42/60
251/251 - 8s - 31ms/step - accuracy: 0.6879 - loss: 0.8499 - val_accuracy:
0.6881 - val_loss: 0.8253 - learning_rate: 1.2500e-04
Epoch 43/60
251/251 - 11s - 43ms/step - accuracy: 0.6957 - loss: 0.8390 - val_accuracy:
0.6890 - val_loss: 0.8445 - learning_rate: 1.2500e-04
Epoch 1/60
251/251 - 10s - 39ms/step - accuracy: 0.3886 - loss: 1.3558 - val_accuracy:
0.5602 - val_loss: 1.1930 - learning_rate: 5.0000e-04
Epoch 2/60
251/251 - 8s - 30ms/step - accuracy: 0.5128 - loss: 1.3078 - val_accuracy:
0.5992 - val_loss: 1.1517 - learning_rate: 5.0000e-04
Epoch 3/60
251/251 - 12s - 48ms/step - accuracy: 0.5062 - loss: 1.3048 - val_accuracy:
0.6051 - val_loss: 1.1512 - learning_rate: 5.0000e-04
Epoch 4/60
251/251 - 9s - 34ms/step - accuracy: 0.5286 - loss: 1.2828 - val_accuracy:
0.5522 - val_loss: 1.2117 - learning_rate: 5.0000e-04
Epoch 5/60
251/251 - 8s - 30ms/step - accuracy: 0.5416 - loss: 1.2768 - val_accuracy:
0.6039 - val_loss: 1.1528 - learning_rate: 5.0000e-04
Epoch 6/60
251/251 - 7s - 30ms/step - accuracy: 0.5380 - loss: 1.2697 - val_accuracy:
0.5216 - val_loss: 1.2254 - learning_rate: 5.0000e-04
Epoch 7/60
251/251 - 11s - 43ms/step - accuracy: 0.5432 - loss: 1.2518 - val_accuracy:
0.6128 - val_loss: 1.1189 - learning_rate: 2.5000e-04
Epoch 8/60
251/251 - 9s - 36ms/step - accuracy: 0.5492 - loss: 1.2489 - val_accuracy:
0.5864 - val_loss: 1.1278 - learning_rate: 2.5000e-04
Epoch 9/60
251/251 - 8s - 33ms/step - accuracy: 0.5372 - loss: 1.2483 - val_accuracy:
0.5929 - val_loss: 1.1131 - learning_rate: 2.5000e-04
Epoch 10/60

251/251 - 8s - 31ms/step - accuracy: 0.5560 - loss: 1.2342 - val_accuracy:
0.6021 - val_loss: 1.1070 - learning_rate: 2.5000e-04
Epoch 11/60
251/251 - 11s - 43ms/step - accuracy: 0.5621 - loss: 1.2260 - val_accuracy:
0.6012 - val_loss: 1.0915 - learning_rate: 2.5000e-04
Epoch 12/60
251/251 - 8s - 31ms/step - accuracy: 0.5435 - loss: 1.2179 - val_accuracy:
0.6081 - val_loss: 1.0416 - learning_rate: 2.5000e-04
Epoch 13/60
251/251 - 8s - 32ms/step - accuracy: 0.5436 - loss: 1.2086 - val_accuracy:
0.5792 - val_loss: 1.0765 - learning_rate: 2.5000e-04
Epoch 14/60
251/251 - 9s - 35ms/step - accuracy: 0.5567 - loss: 1.1915 - val_accuracy:
0.6140 - val_loss: 1.0665 - learning_rate: 2.5000e-04
Epoch 15/60
251/251 - 11s - 43ms/step - accuracy: 0.5619 - loss: 1.1835 - val_accuracy:
0.6069 - val_loss: 1.0397 - learning_rate: 2.5000e-04
Epoch 16/60
251/251 - 8s - 31ms/step - accuracy: 0.5682 - loss: 1.1583 - val_accuracy:
0.6090 - val_loss: 1.0263 - learning_rate: 2.5000e-04
Epoch 17/60
251/251 - 8s - 30ms/step - accuracy: 0.5680 - loss: 1.1452 - val_accuracy:
0.6384 - val_loss: 0.9722 - learning_rate: 2.5000e-04
Epoch 18/60
251/251 - 8s - 30ms/step - accuracy: 0.5734 - loss: 1.1310 - val_accuracy:
0.6331 - val_loss: 0.9812 - learning_rate: 2.5000e-04
Epoch 19/60
251/251 - 13s - 51ms/step - accuracy: 0.5733 - loss: 1.1245 - val_accuracy:
0.5944 - val_loss: 1.0117 - learning_rate: 2.5000e-04
Epoch 20/60
251/251 - 8s - 32ms/step - accuracy: 0.5819 - loss: 1.1147 - val_accuracy:
0.6441 - val_loss: 0.9438 - learning_rate: 2.5000e-04
Epoch 21/60
251/251 - 8s - 31ms/step - accuracy: 0.5752 - loss: 1.1254 - val_accuracy:
0.6450 - val_loss: 0.9331 - learning_rate: 2.5000e-04
Epoch 22/60
251/251 - 8s - 31ms/step - accuracy: 0.5894 - loss: 1.0981 - val_accuracy:
0.6482 - val_loss: 0.9189 - learning_rate: 2.5000e-04
Epoch 23/60
251/251 - 10s - 40ms/step - accuracy: 0.5937 - loss: 1.0875 - val_accuracy:
0.6500 - val_loss: 0.9323 - learning_rate: 2.5000e-04
Epoch 24/60
251/251 - 9s - 36ms/step - accuracy: 0.6005 - loss: 1.0860 - val_accuracy:
0.6530 - val_loss: 0.9019 - learning_rate: 2.5000e-04
Epoch 25/60
251/251 - 10s - 38ms/step - accuracy: 0.5988 - loss: 1.0849 - val_accuracy:
0.6435 - val_loss: 0.9460 - learning_rate: 2.5000e-04
Epoch 26/60
251/251 - 8s - 31ms/step - accuracy: 0.6040 - loss: 1.0691 - val_accuracy:
0.6488 - val_loss: 0.9258 - learning_rate: 2.5000e-04

Epoch 27/60
251/251 - 11s - 43ms/step - accuracy: 0.6090 - loss: 1.0638 - val_accuracy:
0.6705 - val_loss: 0.8789 - learning_rate: 2.5000e-04
Epoch 28/60
251/251 - 8s - 30ms/step - accuracy: 0.6079 - loss: 1.0584 - val_accuracy:
0.6533 - val_loss: 0.9264 - learning_rate: 2.5000e-04
Epoch 29/60
251/251 - 9s - 37ms/step - accuracy: 0.6142 - loss: 1.0533 - val_accuracy:
0.6580 - val_loss: 0.9112 - learning_rate: 2.5000e-04
Epoch 30/60
251/251 - 8s - 34ms/step - accuracy: 0.6185 - loss: 1.0412 - val_accuracy:
0.6521 - val_loss: 0.8926 - learning_rate: 2.5000e-04
Epoch 31/60
251/251 - 11s - 44ms/step - accuracy: 0.6172 - loss: 1.0301 - val_accuracy:
0.6664 - val_loss: 0.8952 - learning_rate: 1.2500e-04
Epoch 32/60
251/251 - 8s - 31ms/step - accuracy: 0.6234 - loss: 1.0233 - val_accuracy:
0.6774 - val_loss: 0.8670 - learning_rate: 1.2500e-04
Epoch 33/60
251/251 - 8s - 31ms/step - accuracy: 0.6318 - loss: 1.0145 - val_accuracy:
0.6789 - val_loss: 0.8526 - learning_rate: 1.2500e-04
Epoch 34/60
251/251 - 9s - 35ms/step - accuracy: 0.6269 - loss: 1.0181 - val_accuracy:
0.6589 - val_loss: 0.8965 - learning_rate: 1.2500e-04
Epoch 35/60
251/251 - 11s - 44ms/step - accuracy: 0.6354 - loss: 1.0077 - val_accuracy:
0.6735 - val_loss: 0.8784 - learning_rate: 1.2500e-04
Epoch 36/60
251/251 - 8s - 30ms/step - accuracy: 0.6351 - loss: 1.0157 - val_accuracy:
0.6589 - val_loss: 0.9060 - learning_rate: 1.2500e-04
Epoch 37/60
251/251 - 8s - 30ms/step - accuracy: 0.6348 - loss: 1.0053 - val_accuracy:
0.6679 - val_loss: 0.8878 - learning_rate: 6.2500e-05
Epoch 38/60
251/251 - 8s - 31ms/step - accuracy: 0.6316 - loss: 0.9964 - val_accuracy:
0.6661 - val_loss: 0.8723 - learning_rate: 6.2500e-05
Epoch 39/60
251/251 - 11s - 44ms/step - accuracy: 0.6383 - loss: 0.9971 - val_accuracy:
0.6798 - val_loss: 0.8480 - learning_rate: 6.2500e-05
Epoch 40/60
251/251 - 9s - 36ms/step - accuracy: 0.6369 - loss: 0.9974 - val_accuracy:
0.6676 - val_loss: 0.8952 - learning_rate: 6.2500e-05
Epoch 41/60
251/251 - 8s - 30ms/step - accuracy: 0.6424 - loss: 0.9917 - val_accuracy:
0.6774 - val_loss: 0.8585 - learning_rate: 6.2500e-05
Epoch 42/60
251/251 - 8s - 30ms/step - accuracy: 0.6413 - loss: 0.9875 - val_accuracy:
0.6661 - val_loss: 0.8806 - learning_rate: 6.2500e-05
Epoch 43/60
251/251 - 11s - 43ms/step - accuracy: 0.6403 - loss: 0.9848 - val_accuracy:

0.6726 - val_loss: 0.8780 - learning_rate: 3.1250e-05
Epoch 44/60
251/251 - 8s - 30ms/step - accuracy: 0.6470 - loss: 0.9801 - val_accuracy:
0.6747 - val_loss: 0.8760 - learning_rate: 3.1250e-05
Epoch 45/60
251/251 - 10s - 38ms/step - accuracy: 0.6445 - loss: 0.9803 - val_accuracy:
0.6812 - val_loss: 0.8568 - learning_rate: 3.1250e-05
Epoch 46/60
251/251 - 8s - 32ms/step - accuracy: 0.6434 - loss: 0.9717 - val_accuracy:
0.6735 - val_loss: 0.8754 - learning_rate: 1.5625e-05
Epoch 47/60
251/251 - 11s - 43ms/step - accuracy: 0.6440 - loss: 0.9764 - val_accuracy:
0.6756 - val_loss: 0.8705 - learning_rate: 1.5625e-05
Epoch 1/60
251/251 - 10s - 39ms/step - accuracy: 0.3961 - loss: 1.3602 - val_accuracy:
0.4912 - val_loss: 1.3394 - learning_rate: 0.0010
Epoch 2/60
251/251 - 8s - 30ms/step - accuracy: 0.4984 - loss: 1.3339 - val_accuracy:
0.4410 - val_loss: 1.3634 - learning_rate: 0.0010
Epoch 3/60
251/251 - 9s - 37ms/step - accuracy: 0.4206 - loss: 1.3565 - val_accuracy:
0.1689 - val_loss: 1.4384 - learning_rate: 0.0010
Epoch 4/60
251/251 - 11s - 44ms/step - accuracy: 0.4690 - loss: 1.3395 - val_accuracy:
0.5055 - val_loss: 1.2612 - learning_rate: 0.0010
Epoch 5/60
251/251 - 8s - 31ms/step - accuracy: 0.4858 - loss: 1.3276 - val_accuracy:
0.5911 - val_loss: 1.1688 - learning_rate: 0.0010
Epoch 6/60
251/251 - 8s - 31ms/step - accuracy: 0.4691 - loss: 1.3272 - val_accuracy:
0.3940 - val_loss: 1.3006 - learning_rate: 0.0010
Epoch 7/60
251/251 - 8s - 31ms/step - accuracy: 0.4106 - loss: 1.3525 - val_accuracy:
0.5742 - val_loss: 1.2387 - learning_rate: 0.0010
Epoch 8/60
251/251 - 12s - 47ms/step - accuracy: 0.4929 - loss: 1.3162 - val_accuracy:
0.5617 - val_loss: 1.1751 - learning_rate: 0.0010
Epoch 9/60
251/251 - 8s - 30ms/step - accuracy: 0.5014 - loss: 1.3028 - val_accuracy:
0.5822 - val_loss: 1.2048 - learning_rate: 5.0000e-04
Epoch 10/60
251/251 - 8s - 30ms/step - accuracy: 0.5245 - loss: 1.2927 - val_accuracy:
0.6036 - val_loss: 1.1465 - learning_rate: 5.0000e-04
Epoch 11/60
251/251 - 8s - 30ms/step - accuracy: 0.5139 - loss: 1.2893 - val_accuracy:
0.5822 - val_loss: 1.1868 - learning_rate: 5.0000e-04
Epoch 12/60
251/251 - 11s - 43ms/step - accuracy: 0.5231 - loss: 1.2775 - val_accuracy:
0.5742 - val_loss: 1.2027 - learning_rate: 5.0000e-04
Epoch 13/60

251/251 - 8s - 34ms/step - accuracy: 0.5255 - loss: 1.2749 - val_accuracy:
0.5876 - val_loss: 1.1714 - learning_rate: 5.0000e-04
Epoch 14/60
251/251 - 9s - 37ms/step - accuracy: 0.5249 - loss: 1.2627 - val_accuracy:
0.6015 - val_loss: 1.1441 - learning_rate: 2.5000e-04
Epoch 15/60
251/251 - 8s - 30ms/step - accuracy: 0.5271 - loss: 1.2588 - val_accuracy:
0.6048 - val_loss: 1.1177 - learning_rate: 2.5000e-04
Epoch 16/60
251/251 - 11s - 43ms/step - accuracy: 0.5314 - loss: 1.2542 - val_accuracy:
0.6063 - val_loss: 1.1306 - learning_rate: 2.5000e-04
Epoch 17/60
251/251 - 8s - 31ms/step - accuracy: 0.5374 - loss: 1.2471 - val_accuracy:
0.6152 - val_loss: 1.0939 - learning_rate: 2.5000e-04
Epoch 18/60
251/251 - 8s - 31ms/step - accuracy: 0.5367 - loss: 1.2467 - val_accuracy:
0.6125 - val_loss: 1.0959 - learning_rate: 2.5000e-04
Epoch 19/60
251/251 - 9s - 35ms/step - accuracy: 0.5404 - loss: 1.2443 - val_accuracy:
0.5953 - val_loss: 1.1365 - learning_rate: 2.5000e-04
Epoch 20/60
251/251 - 11s - 43ms/step - accuracy: 0.5421 - loss: 1.2382 - val_accuracy:
0.6096 - val_loss: 1.1044 - learning_rate: 2.5000e-04
Epoch 21/60
251/251 - 8s - 31ms/step - accuracy: 0.5422 - loss: 1.2369 - val_accuracy:
0.6084 - val_loss: 1.1015 - learning_rate: 1.2500e-04
Epoch 22/60
251/251 - 7s - 30ms/step - accuracy: 0.5454 - loss: 1.2342 - val_accuracy:
0.6099 - val_loss: 1.0997 - learning_rate: 1.2500e-04
Epoch 23/60
251/251 - 8s - 30ms/step - accuracy: 0.5461 - loss: 1.2283 - val_accuracy:
0.6063 - val_loss: 1.1033 - learning_rate: 1.2500e-04
Epoch 24/60
251/251 - 12s - 48ms/step - accuracy: 0.5531 - loss: 1.2283 - val_accuracy:
0.6206 - val_loss: 1.0868 - learning_rate: 6.2500e-05
Epoch 25/60
251/251 - 8s - 30ms/step - accuracy: 0.5554 - loss: 1.2241 - val_accuracy:
0.6134 - val_loss: 1.1005 - learning_rate: 6.2500e-05
Epoch 26/60
251/251 - 8s - 31ms/step - accuracy: 0.5491 - loss: 1.2202 - val_accuracy:
0.6152 - val_loss: 1.0811 - learning_rate: 6.2500e-05
Epoch 27/60
251/251 - 8s - 30ms/step - accuracy: 0.5469 - loss: 1.2157 - val_accuracy:
0.6114 - val_loss: 1.0982 - learning_rate: 6.2500e-05
Epoch 28/60
251/251 - 8s - 30ms/step - accuracy: 0.5494 - loss: 1.2186 - val_accuracy:
0.6045 - val_loss: 1.1024 - learning_rate: 6.2500e-05
Epoch 29/60
251/251 - 12s - 49ms/step - accuracy: 0.5539 - loss: 1.2103 - val_accuracy:
0.6137 - val_loss: 1.0809 - learning_rate: 6.2500e-05

Epoch 30/60
251/251 - 9s - 35ms/step - accuracy: 0.5497 - loss: 1.2098 - val_accuracy:
0.6158 - val_loss: 1.0781 - learning_rate: 6.2500e-05
Epoch 31/60
251/251 - 8s - 30ms/step - accuracy: 0.5521 - loss: 1.2078 - val_accuracy:
0.6123 - val_loss: 1.0797 - learning_rate: 6.2500e-05
Epoch 32/60
251/251 - 11s - 44ms/step - accuracy: 0.5497 - loss: 1.2065 - val_accuracy:
0.6164 - val_loss: 1.0635 - learning_rate: 6.2500e-05
Epoch 33/60
251/251 - 8s - 30ms/step - accuracy: 0.5492 - loss: 1.2079 - val_accuracy:
0.6072 - val_loss: 1.0677 - learning_rate: 6.2500e-05
Epoch 34/60
251/251 - 8s - 33ms/step - accuracy: 0.5517 - loss: 1.2047 - val_accuracy:
0.5950 - val_loss: 1.0897 - learning_rate: 6.2500e-05
Epoch 35/60
251/251 - 9s - 35ms/step - accuracy: 0.5546 - loss: 1.1990 - val_accuracy:
0.6063 - val_loss: 1.0681 - learning_rate: 6.2500e-05
Epoch 36/60
251/251 - 11s - 43ms/step - accuracy: 0.5586 - loss: 1.1992 - val_accuracy:
0.6131 - val_loss: 1.0578 - learning_rate: 3.1250e-05
Epoch 37/60
251/251 - 8s - 31ms/step - accuracy: 0.5569 - loss: 1.1960 - val_accuracy:
0.6063 - val_loss: 1.0747 - learning_rate: 3.1250e-05
Epoch 38/60
251/251 - 8s - 31ms/step - accuracy: 0.5578 - loss: 1.1866 - val_accuracy:
0.6182 - val_loss: 1.0463 - learning_rate: 3.1250e-05
Epoch 39/60
251/251 - 8s - 31ms/step - accuracy: 0.5567 - loss: 1.1919 - val_accuracy:
0.6161 - val_loss: 1.0579 - learning_rate: 3.1250e-05
Epoch 40/60
251/251 - 12s - 48ms/step - accuracy: 0.5573 - loss: 1.1950 - val_accuracy:
0.6146 - val_loss: 1.0555 - learning_rate: 3.1250e-05
Epoch 41/60
251/251 - 8s - 30ms/step - accuracy: 0.5537 - loss: 1.1950 - val_accuracy:
0.6185 - val_loss: 1.0472 - learning_rate: 3.1250e-05
Epoch 42/60
251/251 - 8s - 30ms/step - accuracy: 0.5599 - loss: 1.1940 - val_accuracy:
0.6152 - val_loss: 1.0543 - learning_rate: 1.5625e-05
Epoch 43/60
251/251 - 8s - 30ms/step - accuracy: 0.5576 - loss: 1.1921 - val_accuracy:
0.6209 - val_loss: 1.0448 - learning_rate: 1.5625e-05
Epoch 44/60
251/251 - 8s - 31ms/step - accuracy: 0.5582 - loss: 1.1903 - val_accuracy:
0.6215 - val_loss: 1.0430 - learning_rate: 1.5625e-05
Epoch 45/60
251/251 - 13s - 51ms/step - accuracy: 0.5572 - loss: 1.1931 - val_accuracy:
0.6155 - val_loss: 1.0490 - learning_rate: 1.5625e-05
Epoch 46/60
251/251 - 9s - 34ms/step - accuracy: 0.5600 - loss: 1.1843 - val_accuracy:

```
0.6111 - val_loss: 1.0460 - learning_rate: 1.5625e-05
Epoch 47/60
251/251 - 8s - 30ms/step - accuracy: 0.5606 - loss: 1.1870 - val_accuracy:
0.6191 - val_loss: 1.0438 - learning_rate: 1.5625e-05
Epoch 48/60
251/251 - 11s - 43ms/step - accuracy: 0.5579 - loss: 1.1830 - val_accuracy:
0.6164 - val_loss: 1.0449 - learning_rate: 1.0000e-05
Epoch 49/60
251/251 - 8s - 31ms/step - accuracy: 0.5628 - loss: 1.1826 - val_accuracy:
0.6170 - val_loss: 1.0407 - learning_rate: 1.0000e-05
Epoch 50/60
251/251 - 9s - 36ms/step - accuracy: 0.5567 - loss: 1.1852 - val_accuracy:
0.6188 - val_loss: 1.0369 - learning_rate: 1.0000e-05
Epoch 51/60
251/251 - 8s - 30ms/step - accuracy: 0.5599 - loss: 1.1872 - val_accuracy:
0.6143 - val_loss: 1.0485 - learning_rate: 1.0000e-05
Epoch 52/60
251/251 - 11s - 43ms/step - accuracy: 0.5561 - loss: 1.1844 - val_accuracy:
0.6120 - val_loss: 1.0452 - learning_rate: 1.0000e-05
Epoch 53/60
251/251 - 8s - 31ms/step - accuracy: 0.5621 - loss: 1.1853 - val_accuracy:
0.6146 - val_loss: 1.0409 - learning_rate: 1.0000e-05
Epoch 54/60
251/251 - 8s - 30ms/step - accuracy: 0.5608 - loss: 1.1817 - val_accuracy:
0.6114 - val_loss: 1.0448 - learning_rate: 1.0000e-05
Epoch 55/60
251/251 - 9s - 35ms/step - accuracy: 0.5584 - loss: 1.1802 - val_accuracy:
0.6090 - val_loss: 1.0446 - learning_rate: 1.0000e-05
Epoch 56/60
251/251 - 8s - 30ms/step - accuracy: 0.5556 - loss: 1.1861 - val_accuracy:
0.6105 - val_loss: 1.0437 - learning_rate: 1.0000e-05
Epoch 57/60
251/251 - 11s - 44ms/step - accuracy: 0.5567 - loss: 1.1850 - val_accuracy:
0.6123 - val_loss: 1.0409 - learning_rate: 1.0000e-05
Epoch 58/60
251/251 - 8s - 30ms/step - accuracy: 0.5634 - loss: 1.1832 - val_accuracy:
0.6164 - val_loss: 1.0388 - learning_rate: 1.0000e-05

lstm grid-search results:
      lr  dropout  best_val_acc
0  0.0010      0.3        0.6958
1  0.0005      0.3        0.6812
2  0.0010      0.5        0.6215
Best lstm config: {'lr': 0.001, 'dropout': 0.3} (val_acc=0.6958)
```

[45]:
```
# ---- Run CNN search ----
best_cnn, cnn_grid_table = grid_search(
    build_cnn, CNN_GRID, cnn_train_ds, cnn_val_ds, cw_cnn, tag="cnn")
cnn_model, cnn_history = best_cnn["model"], best_cnn["history"]
```

Epoch 1/60

2026-07-26 01:50:51.126866: W
external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
965869568 exceeds 10% of free system memory.
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1785055861.507166 3363 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'loop_select_fusion_22', 4
bytes spill stores, 12 bytes spill loads
ptxas warning : Registers are spilled to local memory in function
'__cuda_sm3x_div_rn_noftz_f32_slowpath', 4 bytes spill stores, 4 bytes spill
loads

2026-07-26 01:51:10.972941: E
external/local_xla/xla/service/slow_operation_alarm.cc:65] Trying algorithm
eng15{k5=1,k6=0,k7=1,k10=7} for conv (f16[18,128,32,32]{3,2,1,0}, u8[0]{0})
custom-call(f16[18,64,32,32]{3,2,1,0}, f16[128,64,3,3]{3,2,1,0}, f16[128]{0}),
window={size=3x3 pad=1_1x1_1}, dim_labels=bf01_oi01->bf01,
custom_call_target="__cudnn$convBiasActivationForward", backend_config={"operati
on_queue_id":"0","wait_on_operation_queues":[],"cudnn_conv_backend_config":{"con
v_result_scale":1,"activation_mode":"kRelu","side_input_scale":0,"leakyrelu_alph
a":0}} is taking a while…
2026-07-26 01:51:10.976026: E
external/local_xla/xla/service/slow_operation_alarm.cc:133] The operation took
3.030756741s
Trying algorithm eng15{k5=1,k6=0,k7=1,k10=7} for conv
(f16[18,128,32,32]{3,2,1,0}, u8[0]{0}) custom-call(f16[18,64,32,32]{3,2,1,0},
f16[128,64,3,3]{3,2,1,0}, f16[128]{0}), window={size=3x3 pad=1_1x1_1},
dim_labels=bf01_oi01->bf01,
custom_call_target="__cudnn$convBiasActivationForward", backend_config={"operati
on_queue_id":"0","wait_on_operation_queues":[],"cudnn_conv_backend_config":{"con
v_result_scale":1,"activation_mode":"kRelu","side_input_scale":0,"leakyrelu_alph
a":0}} is taking a while…

231/231 - 24s - 106ms/step - accuracy: 0.6033 - loss: 1.0181 - val_accuracy:
0.5245 - val_loss: 2.7206 - learning_rate: 0.0010
Epoch 2/60
231/231 - 6s - 26ms/step - accuracy: 0.7047 - loss: 0.7858 - val_accuracy:
0.1742 - val_loss: 8.2504 - learning_rate: 0.0010
Epoch 3/60
231/231 - 6s - 25ms/step - accuracy: 0.7492 - loss: 0.6711 - val_accuracy:
0.1742 - val_loss: 13.2859 - learning_rate: 0.0010
Epoch 4/60
231/231 - 6s - 25ms/step - accuracy: 0.7778 - loss: 0.5961 - val_accuracy:
0.2152 - val_loss: 7.2081 - learning_rate: 0.0010
Epoch 5/60
231/231 - 6s - 25ms/step - accuracy: 0.8197 - loss: 0.4921 - val_accuracy:
0.6405 - val_loss: 1.2255 - learning_rate: 5.0000e-04
Epoch 6/60
231/231 - 9s - 39ms/step - accuracy: 0.8415 - loss: 0.4404 - val_accuracy:

0.6348 - val_loss: 1.3101 - learning_rate: 5.0000e-04
Epoch 7/60
231/231 - 7s - 28ms/step - accuracy: 0.8525 - loss: 0.4058 - val_accuracy:
0.7524 - val_loss: 0.6617 - learning_rate: 5.0000e-04
Epoch 8/60
231/231 - 6s - 27ms/step - accuracy: 0.8644 - loss: 0.3698 - val_accuracy:
0.6510 - val_loss: 0.9826 - learning_rate: 5.0000e-04
Epoch 9/60
231/231 - 6s - 26ms/step - accuracy: 0.8813 - loss: 0.3326 - val_accuracy:
0.5836 - val_loss: 5.5205 - learning_rate: 5.0000e-04
Epoch 10/60
231/231 - 6s - 26ms/step - accuracy: 0.8878 - loss: 0.3023 - val_accuracy:
0.6319 - val_loss: 0.9457 - learning_rate: 5.0000e-04
Epoch 11/60
231/231 - 6s - 26ms/step - accuracy: 0.9105 - loss: 0.2515 - val_accuracy:
0.7362 - val_loss: 0.7189 - learning_rate: 2.5000e-04
Epoch 12/60
231/231 - 9s - 37ms/step - accuracy: 0.9176 - loss: 0.2312 - val_accuracy:
0.7441 - val_loss: 0.7362 - learning_rate: 2.5000e-04
Epoch 13/60
231/231 - 6s - 26ms/step - accuracy: 0.9286 - loss: 0.2090 - val_accuracy:
0.7308 - val_loss: 0.7374 - learning_rate: 2.5000e-04
Epoch 14/60
231/231 - 6s - 26ms/step - accuracy: 0.9382 - loss: 0.1805 - val_accuracy:
0.7495 - val_loss: 0.7337 - learning_rate: 1.2500e-04
Epoch 15/60
231/231 - 6s - 26ms/step - accuracy: 0.9413 - loss: 0.1718 - val_accuracy:
0.7492 - val_loss: 0.7555 - learning_rate: 1.2500e-04
Epoch 1/60
231/231 - 17s - 76ms/step - accuracy: 0.5895 - loss: 1.0500 - val_accuracy:
0.3916 - val_loss: 1.4575 - learning_rate: 5.0000e-04
Epoch 2/60
231/231 - 6s - 27ms/step - accuracy: 0.7002 - loss: 0.8167 - val_accuracy:
0.1758 - val_loss: 3.3224 - learning_rate: 5.0000e-04
Epoch 3/60
231/231 - 6s - 26ms/step - accuracy: 0.7316 - loss: 0.7191 - val_accuracy:
0.4615 - val_loss: 2.2692 - learning_rate: 5.0000e-04
Epoch 4/60
231/231 - 6s - 26ms/step - accuracy: 0.7559 - loss: 0.6501 - val_accuracy:
0.1993 - val_loss: 5.0956 - learning_rate: 5.0000e-04
Epoch 5/60
231/231 - 6s - 26ms/step - accuracy: 0.7898 - loss: 0.5686 - val_accuracy:
0.2565 - val_loss: 2.4132 - learning_rate: 2.5000e-04
Epoch 6/60
231/231 - 9s - 37ms/step - accuracy: 0.8031 - loss: 0.5329 - val_accuracy:
0.7670 - val_loss: 0.6217 - learning_rate: 2.5000e-04
Epoch 7/60
231/231 - 6s - 26ms/step - accuracy: 0.8135 - loss: 0.5004 - val_accuracy:
0.1961 - val_loss: 4.0303 - learning_rate: 2.5000e-04
Epoch 8/60

231/231 - 6s - 26ms/step - accuracy: 0.8308 - loss: 0.4671 - val_accuracy:
0.5099 - val_loss: 1.3035 - learning_rate: 2.5000e-04
Epoch 9/60
231/231 - 6s - 26ms/step - accuracy: 0.8389 - loss: 0.4469 - val_accuracy:
0.6065 - val_loss: 1.1688 - learning_rate: 2.5000e-04
Epoch 10/60
231/231 - 6s - 27ms/step - accuracy: 0.8586 - loss: 0.4012 - val_accuracy:
0.7854 - val_loss: 0.5668 - learning_rate: 1.2500e-04
Epoch 11/60
231/231 - 10s - 41ms/step - accuracy: 0.8678 - loss: 0.3745 - val_accuracy:
0.7565 - val_loss: 0.6487 - learning_rate: 1.2500e-04
Epoch 12/60
231/231 - 6s - 27ms/step - accuracy: 0.8684 - loss: 0.3704 - val_accuracy:
0.6736 - val_loss: 0.9541 - learning_rate: 1.2500e-04
Epoch 13/60
231/231 - 6s - 27ms/step - accuracy: 0.8749 - loss: 0.3541 - val_accuracy:
0.7896 - val_loss: 0.5680 - learning_rate: 1.2500e-04
Epoch 14/60
231/231 - 6s - 26ms/step - accuracy: 0.8857 - loss: 0.3305 - val_accuracy:
0.7810 - val_loss: 0.5991 - learning_rate: 6.2500e-05
Epoch 15/60
231/231 - 6s - 26ms/step - accuracy: 0.8902 - loss: 0.3174 - val_accuracy:
0.7918 - val_loss: 0.5721 - learning_rate: 6.2500e-05
Epoch 16/60
231/231 - 9s - 39ms/step - accuracy: 0.8960 - loss: 0.3097 - val_accuracy:
0.7873 - val_loss: 0.5829 - learning_rate: 6.2500e-05
Epoch 17/60
231/231 - 6s - 27ms/step - accuracy: 0.8973 - loss: 0.3025 - val_accuracy:
0.8026 - val_loss: 0.5505 - learning_rate: 3.1250e-05
Epoch 18/60
231/231 - 6s - 26ms/step - accuracy: 0.9002 - loss: 0.2933 - val_accuracy:
0.7635 - val_loss: 0.6513 - learning_rate: 3.1250e-05
Epoch 19/60
231/231 - 6s - 26ms/step - accuracy: 0.9030 - loss: 0.2909 - val_accuracy:
0.7549 - val_loss: 0.6685 - learning_rate: 3.1250e-05
Epoch 20/60
231/231 - 6s - 27ms/step - accuracy: 0.8996 - loss: 0.2904 - val_accuracy:
0.7762 - val_loss: 0.6085 - learning_rate: 3.1250e-05
Epoch 21/60
231/231 - 9s - 40ms/step - accuracy: 0.9077 - loss: 0.2774 - val_accuracy:
0.7893 - val_loss: 0.5902 - learning_rate: 1.5625e-05
Epoch 22/60
231/231 - 6s - 28ms/step - accuracy: 0.9076 - loss: 0.2785 - val_accuracy:
0.7705 - val_loss: 0.6386 - learning_rate: 1.5625e-05
Epoch 23/60
231/231 - 6s - 26ms/step - accuracy: 0.9042 - loss: 0.2794 - val_accuracy:
0.7781 - val_loss: 0.6107 - learning_rate: 1.5625e-05
Epoch 24/60
231/231 - 6s - 26ms/step - accuracy: 0.9071 - loss: 0.2745 - val_accuracy:
0.7950 - val_loss: 0.5683 - learning_rate: 1.0000e-05

Epoch 25/60
231/231 - 6s - 26ms/step - accuracy: 0.9097 - loss: 0.2703 - val_accuracy:
0.7870 - val_loss: 0.5757 - learning_rate: 1.0000e-05
Epoch 1/60

I0000 00:00:1785056147.139911 3373 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'loop_select_fusion_22', 4
bytes spill stores, 12 bytes spill loads
ptxas warning : Registers are spilled to local memory in function
'__cuda_sm3x_div_rn_noftz_f32_slowpath', 4 bytes spill stores, 4 bytes spill
loads

231/231 - 17s - 75ms/step - accuracy: 0.5629 - loss: 1.0926 - val_accuracy:
0.5490 - val_loss: 1.5870 - learning_rate: 0.0010
Epoch 2/60
231/231 - 6s - 26ms/step - accuracy: 0.6898 - loss: 0.8215 - val_accuracy:
0.1742 - val_loss: 8.2972 - learning_rate: 0.0010
Epoch 3/60
231/231 - 6s - 26ms/step - accuracy: 0.7301 - loss: 0.7204 - val_accuracy:
0.1834 - val_loss: 7.1532 - learning_rate: 0.0010
Epoch 4/60
231/231 - 6s - 26ms/step - accuracy: 0.7562 - loss: 0.6535 - val_accuracy:
0.4447 - val_loss: 2.0146 - learning_rate: 0.0010
Epoch 5/60
231/231 - 6s - 27ms/step - accuracy: 0.7921 - loss: 0.5573 - val_accuracy:
0.7622 - val_loss: 0.6312 - learning_rate: 5.0000e-04
Epoch 6/60
231/231 - 9s - 40ms/step - accuracy: 0.8154 - loss: 0.5045 - val_accuracy:
0.1758 - val_loss: 11.5084 - learning_rate: 5.0000e-04
Epoch 7/60
231/231 - 6s - 27ms/step - accuracy: 0.8214 - loss: 0.4797 - val_accuracy:
0.7276 - val_loss: 0.7033 - learning_rate: 5.0000e-04
Epoch 8/60
231/231 - 6s - 26ms/step - accuracy: 0.8367 - loss: 0.4455 - val_accuracy:
0.5639 - val_loss: 1.1265 - learning_rate: 5.0000e-04
Epoch 9/60
231/231 - 6s - 27ms/step - accuracy: 0.8612 - loss: 0.3839 - val_accuracy:
0.7959 - val_loss: 0.5313 - learning_rate: 2.5000e-04
Epoch 10/60
231/231 - 6s - 27ms/step - accuracy: 0.8676 - loss: 0.3593 - val_accuracy:
0.7330 - val_loss: 0.9629 - learning_rate: 2.5000e-04
Epoch 11/60
231/231 - 9s - 40ms/step - accuracy: 0.8775 - loss: 0.3447 - val_accuracy:
0.7762 - val_loss: 0.6232 - learning_rate: 2.5000e-04
Epoch 12/60
231/231 - 6s - 26ms/step - accuracy: 0.8857 - loss: 0.3226 - val_accuracy:
0.7699 - val_loss: 0.6093 - learning_rate: 2.5000e-04
Epoch 13/60
231/231 - 6s - 26ms/step - accuracy: 0.8945 - loss: 0.3016 - val_accuracy:
0.7940 - val_loss: 0.5704 - learning_rate: 1.2500e-04

```
Epoch 14/60
231/231 - 6s - 26ms/step - accuracy: 0.8965 - loss: 0.2894 - val_accuracy:
0.7285 - val_loss: 0.7904 - learning_rate: 1.2500e-04
Epoch 15/60
231/231 - 6s - 27ms/step - accuracy: 0.9026 - loss: 0.2760 - val_accuracy:
0.7826 - val_loss: 0.6025 - learning_rate: 1.2500e-04
Epoch 16/60
231/231 - 9s - 40ms/step - accuracy: 0.9115 - loss: 0.2631 - val_accuracy:
0.7940 - val_loss: 0.5626 - learning_rate: 6.2500e-05
Epoch 17/60
231/231 - 6s - 26ms/step - accuracy: 0.9147 - loss: 0.2495 - val_accuracy:
0.7962 - val_loss: 0.5809 - learning_rate: 6.2500e-05

cnn grid-search results:
      lr  dropout  best_val_acc
0 0.0010      0.3        0.7524
1 0.0005      0.3        0.8026
2 0.0010      0.5        0.7962
Best cnn config: {'lr': 0.0005, 'dropout': 0.3} (val_acc=0.8026)
```

[46]:
```python
# ----------------------------------------------------------------------
# RECOVERY CELL — run this instead of cells 36–37 to skip retraining.
# Requires: best_<tag>_cfg<i>.keras AND history_<tag>_cfg<i>.json on disk.
# After running this cell, skip straight to Section 11 (plot history).
# ----------------------------------------------------------------------
import json as _json

def load_best_from_disk(tag, grid):
    best_val_acc, best_idx = -1, None
    for i in range(len(grid)):
        hpath = ARTIFACTS_DIR / f"history_{tag}_cfg{i}.json"
        mpath = ARTIFACTS_DIR / f"best_{tag}_cfg{i}.keras"
        if not (hpath.exists() and mpath.exists()):
            print(f"  [{tag}_cfg{i}] missing — skipped")
            continue
        with open(hpath) as fh:
            h = _json.load(fh)
        v = max(h.get("val_accuracy", [0]))
        print(f"  [{tag}_cfg{i}] val_acc={v:.4f}")
        if v > best_val_acc:
            best_val_acc, best_idx = v, i
            best_h = h
    if best_idx is None:
        raise FileNotFoundError(
            f"No complete checkpoints found for '{tag}'. Run the training cells first.")
    model = tf.keras.models.load_model(str(ARTIFACTS_DIR / f"best_{tag}_cfg{best_idx}.
↪keras"))
    class _MockHistory:
        def __init__(self, data): self.history = data
```

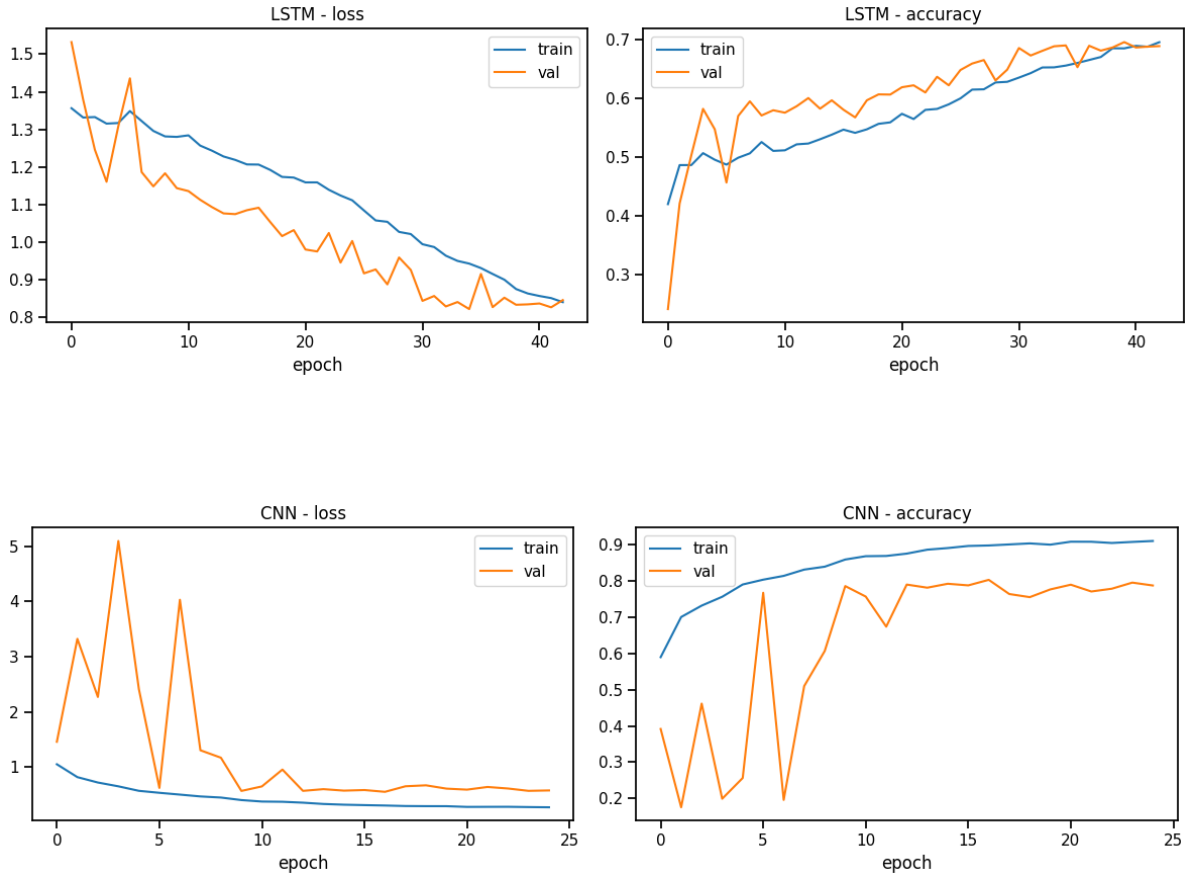
```
    print(f"  → best {tag}: cfg{best_idx}  val_acc={best_val_acc:.4f}")
    return model, _MockHistory(best_h)

print("Loading LSTM ...")
lstm_model, lstm_history = load_best_from_disk("lstm", LSTM_GRID)
print("\nLoading CNN ...")
cnn_model,  cnn_history  = load_best_from_disk("cnn",  CNN_GRID)
print("\nBoth models loaded. Continue from Section 11.")
```

```
Loading LSTM ...
  [lstm_cfg0] val_acc=0.6958
  [lstm_cfg1] val_acc=0.6812
  [lstm_cfg2] val_acc=0.6215
  → best lstm: cfg0  val_acc=0.6958

Loading CNN ...
  [cnn_cfg0] val_acc=0.7524
  [cnn_cfg1] val_acc=0.8026
  [cnn_cfg2] val_acc=0.7962
  → best cnn: cfg1  val_acc=0.8026

Both models loaded. Continue from Section 11.
```

1.13 11. Training and validation curves

The curves below show loss and accuracy for training vs. validation across epochs for each model. A growing gap between the two curves indicates overfitting; flat, low curves indicate underfitting. Interpret these in the report using the figures produced here.

[47]:
```
def plot_history(history, title):
    h = history.history
    fig, ax = plt.subplots(1, 2, figsize=(12, 4))
    ax[0].plot(h["loss"], label="train"); ax[0].plot(h["val_loss"], label="val")
    ax[0].set_title(f"{title} - loss"); ax[0].set_xlabel("epoch"); ax[0].legend()
    ax[1].plot(h["accuracy"], label="train"); ax[1].plot(h["val_accuracy"], label="val")
    ax[1].set_title(f"{title} - accuracy"); ax[1].set_xlabel("epoch"); ax[1].legend()
    plt.tight_layout(); plt.show()

plot_history(lstm_history, "LSTM")
plot_history(cnn_history, "CNN")
```



1.14 12. Evaluation

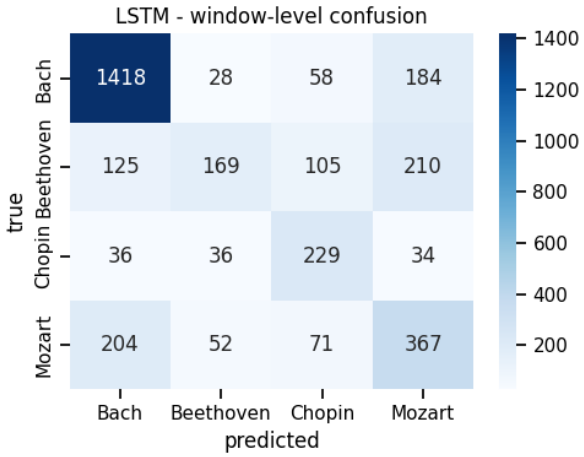
Each model is evaluated on the held-out **test** set at two levels. **Window level** scores every window independently, as in the earlier submission. **Piece level** averages a file's window probability vectors and takes the argmax — one prediction per piece — which is closer to how the model would actually be used, and is only meaningful now that file ids are tracked through the pipeline (Section 6). Macro and weighted averages are reported at window level: **macro** treats every composer equally (fair under imbalance), while **weighted** reflects the test-set class frequencies. Piece level uses macro averages, and adds a confusion matrix alongside the window-level one.

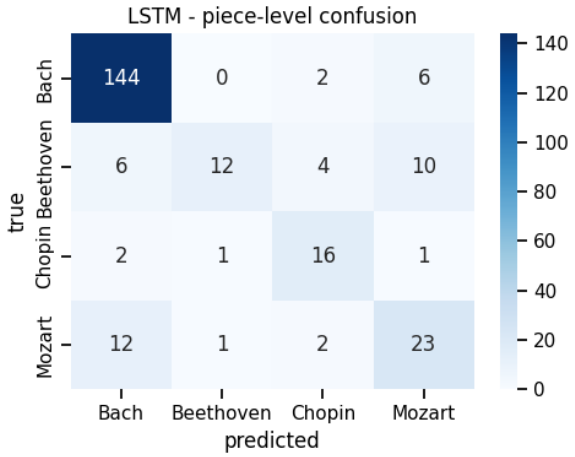
[48]:
```
def evaluate(model, ds, y, file_ids, title):
    proba = model.predict(ds, verbose=0)

    # ---- window level ----
    y_pred_w = proba.argmax(axis=1)
    acc_w = accuracy_score(y, y_pred_w)
    pw_macro, rw_macro, fw_macro, _ = precision_recall_fscore_support(
        y, y_pred_w, average="macro", zero_division=0)
    pw_w, rw_w, fw_w, _ = precision_recall_fscore_support(
        y, y_pred_w, average="weighted", zero_division=0)
```

```
    # ---- piece level: average window probabilities per file, then argmax ----
    pieces = pd.DataFrame(proba)
    pieces["fid"] = file_ids
    pieces["true"] = y
    agg = pieces.groupby("fid").agg({**{i: "mean" for i in range(len(TARGET_COMPOSERS))},
                                     "true": "first"})
    y_true_p = agg["true"].values
    y_pred_p = agg[list(range(len(TARGET_COMPOSERS)))].values.argmax(axis=1)
    acc_p = accuracy_score(y_true_p, y_pred_p)
    pp_macro, rp_macro, fp_macro, _ = precision_recall_fscore_support(
        y_true_p, y_pred_p, average="macro", zero_division=0)

    print(f"===== {title} =====")
    print(f"Window accuracy       : {acc_w:.4f}  (n={len(y)})")
    print(f"Window macro    P/R/F1: {pw_macro:.4f} / {rw_macro:.4f} / {fw_macro:.4f}")
    print(f"Window weighted P/R/F1: {pw_w:.4f} / {rw_w:.4f} / {fw_w:.4f}")
    print(f"Piece  accuracy       : {acc_p:.4f}  (n={len(y_true_p)})")
    print(f"Piece  macro    P/R/F1: {pp_macro:.4f} / {rp_macro:.4f} / {fp_macro:.4f}\n")
    print("Per-class classification report (piece level):")
    print(classification_report(y_true_p, y_pred_p, target_names=TARGET_COMPOSERS,
                                zero_division=0))

    fig, ax = plt.subplots(1, 2, figsize=(10, 4))
    sns.heatmap(confusion_matrix(y, y_pred_w), annot=True, fmt="d", cmap="Blues",
                xticklabels=TARGET_COMPOSERS, yticklabels=TARGET_COMPOSERS, ax=ax[0])
    ax[0].set_title(f"{title} - window-level confusion")
    ax[0].set_xlabel("predicted"); ax[0].set_ylabel("true")
    sns.heatmap(confusion_matrix(y_true_p, y_pred_p), annot=True, fmt="d", cmap="Blues",
                xticklabels=TARGET_COMPOSERS, yticklabels=TARGET_COMPOSERS, ax=ax[1])
    ax[1].set_title(f"{title} - piece-level confusion")
    ax[1].set_xlabel("predicted"); ax[1].set_ylabel("true")
    plt.tight_layout(); plt.show()

    return {"model": title,
            "accuracy_window": acc_w, "f1_macro_window": fw_macro, "f1_weighted_window":␣
↪fw_w,
            "accuracy_piece": acc_p, "precision_macro_piece": pp_macro,
            "recall_macro_piece": rp_macro, "f1_macro_piece": fp_macro}

lstm_metrics = evaluate(lstm_model, seq_test_ds, yseq_test, seq_test_fid, "LSTM")
cnn_metrics  = evaluate(cnn_model,  cnn_test_ds,  ycnn_test,  cnn_test_fid,  "CNN")
```

```
===== LSTM =====
Window accuracy       : 0.6563  (n=3326)
Window macro    P/R/F1: 0.5861 / 0.5825 / 0.5655
Window weighted P/R/F1: 0.6583 / 0.6563 / 0.6446
Piece  accuracy       : 0.8058  (n=242)
Piece  macro    P/R/F1: 0.7442 / 0.6819 / 0.6875

Per-class classification report (piece level):
```

	precision	recall	f1-score	support
Bach	0.88	0.95	0.91	152
Beethoven	0.86	0.38	0.52	32
Chopin	0.67	0.80	0.73	20
Mozart	0.57	0.61	0.59	38
accuracy			0.81	242
macro avg	0.74	0.68	0.69	242
weighted avg	0.81	0.81	0.79	242

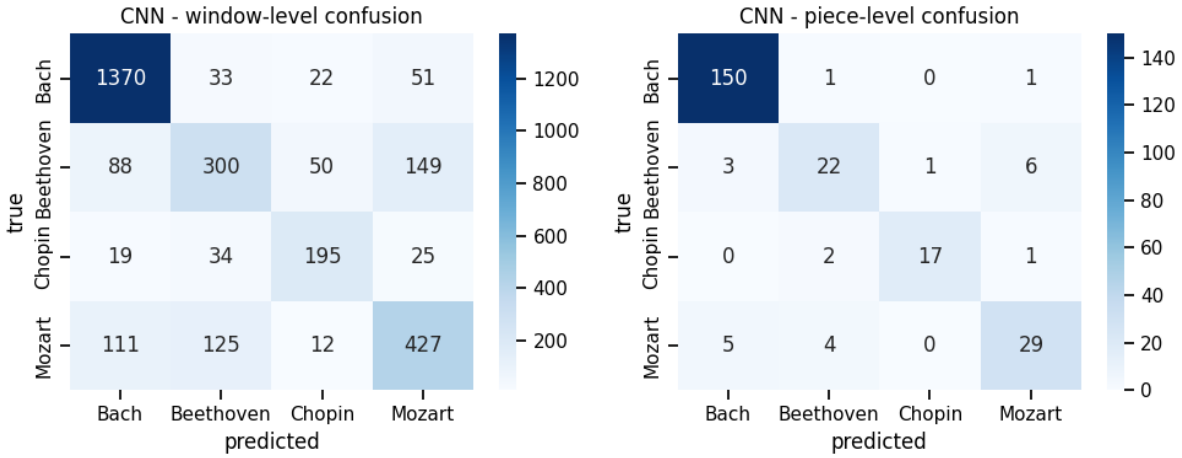




===== CNN =====
Window accuracy : 0.7612 (n=3011)
Window macro P/R/F1: 0.7066 / 0.6965 / 0.7001
Window weighted P/R/F1: 0.7520 / 0.7612 / 0.7551
Piece accuracy : 0.9008 (n=242)
Piece macro P/R/F1: 0.8591 / 0.8219 / 0.8393

Per-class classification report (piece level):

	precision	recall	f1-score	support
Bach	0.95	0.99	0.97	152
Beethoven	0.76	0.69	0.72	32
Chopin	0.94	0.85	0.89	20
Mozart	0.78	0.76	0.77	38
accuracy			0.90	242
macro avg	0.86	0.82	0.84	242
weighted avg	0.90	0.90	0.90	242



1.15 13. Model comparison

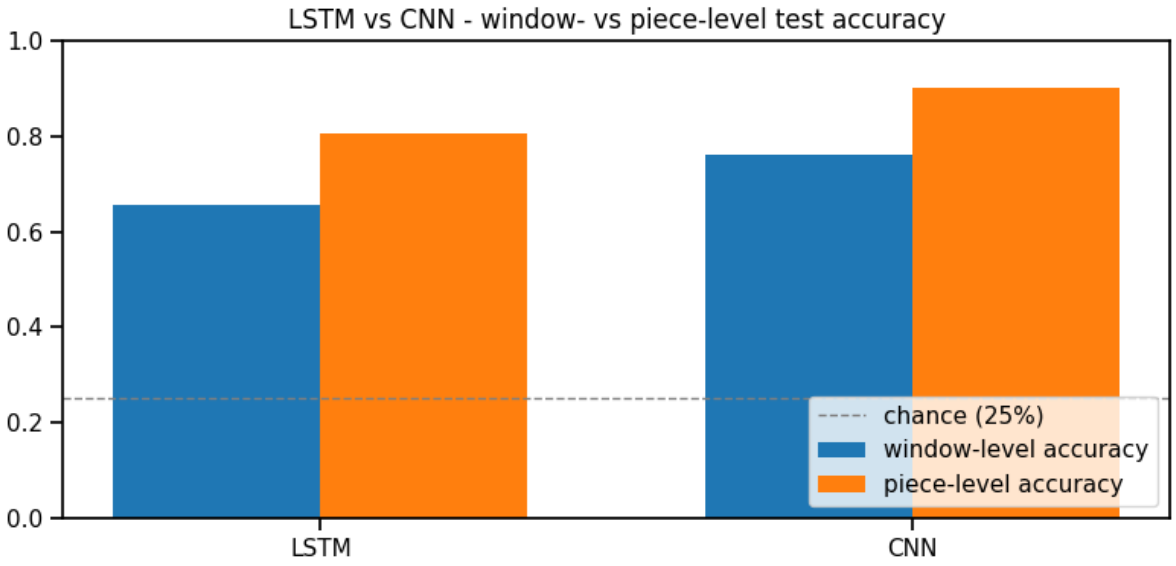
The final comparison table is assembled directly from the metric dictionaries computed above, so it always reflects the real run. Use it as the source for the report's results table.

[49]:
```
comparison = pd.DataFrame([lstm_metrics, cnn_metrics]).set_index("model").round(4)
print("Model comparison (test set):\n")
print(comparison)

fig, ax = plt.subplots(figsize=(8, 4))
x = np.arange(len(comparison)); w = 0.35
ax.bar(x - w/2, comparison["accuracy_window"], w, label="window-level accuracy")
ax.bar(x + w/2, comparison["accuracy_piece"],  w, label="piece-level accuracy")
ax.set_xticks(x); ax.set_xticklabels(comparison.index)
ax.axhline(0.25, ls="--", c="gray", lw=1, label="chance (25%)")
ax.set_ylim(0, 1); ax.legend(loc="lower right")
ax.set_title("LSTM vs CNN - window- vs piece-level test accuracy")
plt.tight_layout(); plt.show()

# Save the comparison so it can be pasted into the report.
comparison.to_csv(VISUALIZATIONS_DIR / "model_comparison.csv")
print(f"\nSaved {VISUALIZATIONS_DIR / 'model_comparison.csv'}")
```

Model comparison (test set):

model	accuracy_window	f1_macro_window	f1_weighted_window	accuracy_piece
LSTM	0.6563	0.5655	0.6446	0.8058
CNN	0.7612	0.7001	0.7551	0.9008

model	precision_macro_piece	recall_macro_piece	f1_macro_piece
LSTM	0.7442	0.6819	0.6875
CNN	0.8591	0.8219	0.8393



Saved build/visualizations/model_comparison.csv